

pateda: Solving optimization problems with Estimation of Distribution Algorithms implemented in Python

Roberto Santana

Department of Computer Science and Artificial Intelligence

University of the Basque Country (UPV/EHU)

roberto.santana@ehu.eus

2026

Abstract

This document describes **pateda**, a Python library for estimation of distribution algorithms (EDAs). **pateda** supports single- and multi-objective optimization over discrete and continuous search spaces. The library provides (i) a set of plug-and-play algorithm classes that encapsulate complete EDA pipelines, (ii) a flexible component-based architecture that allows the user to mix and match any combination of selection, learning, sampling, replacement, and other operators, and (iii) a knowledge-extraction and visualization module for analyzing probabilistic models learned during the evolutionary search. **pateda** builds on the Matlab-based **MATEDA-2.0** package ¹, inheriting its modular philosophy while taking advantage of modern Python scientific computing libraries. It keeps most of the functionalities of **MATEDA-2.0** while incorporating a set of new developments: (i) extensions to discrete problems where variables may have different (binary and non-binary) cardinality, (ii) model learning can explicitly use selection probabilities, (iii) model-learning can incorporate a-priori problem-structure information, (iv) an EDA pipeline search tool for automatic EDA configuration, (v) sampling methods based on most probable configurations, (vi) a dedicated Bayesian Network (BN) learning library (**bayes_nets**²) with over ten BN learning algorithms, (vii) vine-copula models for continuous problems, (viii) mixture of distribution models for discrete and continuous problems. Finally, three specialized libraries that have **pateda** as a dependency have been derived: **perm_pateda**³, for EDAs that solve optimization problems with a permutation-based representation, **nn_pateda**⁴, for EDAs that model search distributions using neural networks, and **latent_search**⁵, for search algorithms that build a latent representation of the optimization problem.

Contents

1	Introduction	4
2	Installation and Package Structure	5
2.1	Installation	5
2.2	Package Structure	5
2.3	Running the Examples	6

¹<https://github.com/rsantana-isg/MATEDA>

²https://github.com/rsantana-isg/bayes_nets

³https://github.com/rsantana-isg/perm_pateda

⁴https://github.com/rsantana-isg/nn_pateda

⁵https://github.com/rsantana-isg/latent_search

3	Defining and Executing an EDA	6
3.1	General Description	6
3.2	The EDA Class	6
3.2.1	Input Parameters	7
3.2.2	Output: Statistics and Cache	7
3.3	The EDAComponents Dataclass	8
3.4	Example: UMDA on the OneMax Problem	9
3.5	Example: Gaussian UMDA on the Sphere Function	9
3.6	Fitness Convention	10
4	Plug-and-Play Algorithm Classes	10
4.1	Discrete Algorithm Classes	10
4.2	Continuous Algorithm Classes	10
4.3	Quick Comparison Example	10
5	Probabilistic Models and Algorithm Families	11
5.1	Discrete Representation	11
5.2	Continuous Representation	11
5.3	Factorized Distributions	11
5.4	Univariate Models	12
5.5	Tree and Forest Models	13
5.6	Bayesian Network Models	13
5.7	Most Probable Configurations	15
5.8	Markov Network Models	16
5.9	Markov Chain Models	16
5.10	Constrained EDAs	17
5.11	Continuous Gaussian Models	17
5.12	Learning from Weighted Solutions	18
6	EDA Components	18
6.1	Seeding Methods	18
6.2	Selection Methods	19
6.3	Replacement Methods	19
6.4	Stopping Conditions	20
6.5	Local Optimization	20
6.5.1	Budget- and subset-aware memetic local search	21
6.6	Exploiting the model structure: network crossover and substructural search	21
6.7	Crossover and Mutation as Learning and Sampling	22
6.8	Repairing Methods	22
7	Test Functions and Benchmark Problems	23
7.1	Discrete Functions	23
7.2	Continuous Functions	24
8	Multi-Objective Optimization	24
8.1	Multi-Objective Problem Definition	24
8.2	Pareto Dominance	25
8.3	Multi-Objective Selection	25
8.4	Metrics, Archive, and Offspring Generation	25
8.5	Multi-Objective EDA Pipeline	26
8.6	Extracting Pareto Fronts	27

9	Knowledge Extraction and Transfer Learning	27
9.1	Dependency Analysis	27
9.2	Fitness Measures	28
9.3	Network Measures and Visualizations	28
9.4	Continuous Variable Analysis	28
9.5	Transfer Learning	29
10	Injecting A-Priori Knowledge	30
10.1	Fixed Factorizations	31
10.2	Interaction Matrices	32
10.3	Biased Initialization	32
10.4	Constrained Search	33
11	Extensions to pateda	33
11.1	Problems with permutation representation (<code>perm_pateda</code>)	33
11.2	Model learning with neural networks (<code>nn_pateda</code>)	34
11.3	Searching in the neural latent representation space (<code>latent_search</code>)	34
12	EDA Pipelines: model operators and a generative grammar	34
12.1	Model operators: MODMOD and MODCONV	34
12.2	A context-free grammar for feasible pipelines	35
12.3	A multi-objective meta-optimizer over pipelines	35
13	Conclusions	35

1 Introduction

Estimation of distribution algorithms (EDAs) [43, 48, 56, 64] are a class of evolutionary algorithms that replace the variation operators of traditional genetic algorithms (GAs) [26, 34] with the explicit estimation and sampling of probabilistic models. Traditional EDAs learn and exploit probabilistic graphical models (PGMs) [39, 40, 45] but other ways of representing and sampling sampling probabilistic distributions, most notably neural networks, have been also proposed [24, 52].

At each generation, a probabilistic model is learned from a selected subset of the current population; new individuals are then generated by sampling from this model. The probabilistic model captures the statistical dependencies between the problem variables that tend to co-occur in high-quality solutions, endowing EDAs with the ability to exploit problem structure in a principled way.

Several classes of probabilistic models have been proposed for EDAs. They range from simple univariate (independent-variable) models [56, 3] through tree-structured models [5, 63] and Bayesian networks [23, 62, 60] to Markov networks [66, 67, 84, 85], factorized distributions [55, 29, 59], and mixtures of distributions [9, 11, 79]. For continuous optimization, Gaussian EDAs [42, 41, 8, 7] and mixture-of-Gaussian models are the most common choices.

Beyond optimization, the probabilistic models learned by EDAs encode information about the problem structure that can be extracted and analyzed. This *knowledge extraction* perspective has been pursued in a number of works [21, 22, 31, 46, 72] and is an important feature of *pateda*. Furthermore, EDAs are a unique arena to test methods for learning PGMs since in this scenarios probabilistic models are learning intensively and the samples from which a current model is learned had been previously generated by another model. Therefore, learning methods for PGMs are expected to be fast, and ideal learning methods should be able to reuse information from previous generations.

pateda is a Python 3 library that implements a wide range of EDAs for discrete and continuous single- and multi-objective optimization. It follows the design principles of *MATEDA-2.0* [74], a Matlab suite with a similar modular philosophy, originally conceived for the design of flexible EDAs. The main design goals of *pateda* are:

- Plug-and-play convenience: Algorithm classes such as *UMDA*, *EBNA*, *GaussianUMDA*, and *GaussianNetworkEDA* provide a one-liner interface for the most common EDA configurations.
- Flexible component assembly: The *EDAComponents* dataclass allows the user to compose any EDA from independently swappable components: seeding, selection, learning, sampling, replacement, mutation, crossover, local optimization, repairing, statistics, and stopping condition.
- Extensibility: Every component type is defined as an abstract base class. Adding a new learning algorithm, selection scheme, or test function requires only the implementation of the corresponding abstract method.
- Single and multi-objective support: The fitness function may return a scalar or a vector. Multi-objective selection (Pareto non-dominated sorting, crowding distance, indicator-based), decomposition (MOEA/D), and archive management are provided.
- Knowledge extraction: A dedicated module provides tools for analyzing the structure of the learned models, computing dependency measures, and visualizing network structures.

The remainder of this paper is organized as follows. Section 2 describes installation and the package structure. Section 3 explains how to define and execute a generic EDA using the core EDA class and *EDAComponents*. Section 4 surveys the plug-and-play algorithm classes. Section 5

details the probabilistic model families, organized by algorithm family. Section 6 documents all EDA component types and their implementations. Section 7 lists the built-in benchmark functions and test problems. Section 8 covers the multi-objective infrastructure. Section 9 describes the knowledge-extraction module. Section 10 explains mechanisms for injecting a-priori structural information.

2 Installation and Package Structure

2.1 Installation

`pateda` is distributed as a Python package located in the `packages/pateda` subdirectory of the repository. Install the most recent version using:

```
1 pip3 install pateda
```

Required dependencies: `numpy`, `scipy`, `scikit-learn`, `networkx`, `matplotlib`, `bayes_nets`. Optional: `pyvinecopulib` (vine-copula models), `seaborn` (visualization).

2.2 Package Structure

After installation, the top-level `pateda` namespace provides direct access to the most commonly used classes:

```
1 from pateda import (
2     EDA, EDAComponents,           # core
3     UMDA, TreeEDA, EBNA, BOA,     # discrete plug-and-play
4     MNFDA, MNFDAG, MOA, FDA,      # Markov network / FDA
5     MKEDA, MTED, AffEDA,          # chain / mixture / affinity
6     CUMDA, CFDA, PBIL,            # constrained / incremental
7     GaussianUMDA, GaussianFull,   # continuous
8     GaussianNetworkEDA,           # Gaussian network
9     MixtureGaussianEDA,           # mixture of Gaussians
10    VineEDA,                       # vine copulas
11 )
```

The internal module tree is:

Module	Contents
core/	EDA executor, EDAComponents, Statistics, Cache, abstract component interfaces, model containers
algorithms/	Plug-and-play wrappers: <code>discrete.py</code> , <code>continuous.py</code> , <code>base.py</code>
learning/	All learning method implementations
sampling/	All sampling method implementations
selection/	Selection operators
replacement/	Replacement operators
seeding/	Population initialization methods
mutation/	Mutation operators
crossover/	Crossover operators
repairing/	Constraint repair methods
local_optimization/	Local search methods
inference/	MAP and k-MAP inference
multiobjective/	Dominance, crowding, MOEA/D, indicators, archive
stop_conditions/	Stopping criteria
statistics/	Population statistics and tracker
functions/	Benchmark functions (discrete and continuous)
knowledge_extraction/	Dependency analysis, network measures, visualizations

2.3 Running the Examples

The `examples/` directory contains runnable scripts for all major algorithm families. For example:

```

1 %cd packages/pateda/examples
2 python3 examples/umda_onemax.py
3 python3 examples/ebna_deceptive.py
4 python3 examples/gaussian_umda_sphere.py
5 python3 examples/tree_eda_ising.py
6 python3 examples/multiobjective_approaches_demo.py

```

3 Defining and Executing an EDA

3.1 General Description

The main component of pateda is the EDA class, defined in `core/eda.py`. It implements the generic EDA loop described in Algorithm 1, analogous to `RunEDA.m` in MATEDA-2.0.

3.2 The EDA Class

An EDA is instantiated with:

```

1 from pateda.core.eda import EDA
2
3 eda = EDA(
4     pop_size      = 300,
5     n_vars        = 20,
6     fitness_func  = my_func,
7     cardinality   = np.full(20, 2),    # binary variables
8     components    = my_components,
9     random_seed   = 42,
10 )

```

Algorithm 1 Estimation of Distribution Algorithm

```
1: Set  $t \leftarrow 0$ 
2: repeat
3:   if  $t = 0$  then
4:     Generate initial population  $D_0$  using the seeding method
5:     Apply repairing method to  $D_0$  (if required)
6:     Evaluate  $D_0$  using the fitness function
7:     Apply local optimization to  $D_0$  (if required)
8:   else
9:     Sample  $D_{\text{new}}$  from the model using the sampling method
10:    Apply mutation to  $D_{\text{new}}$  (if required)
11:    Apply repairing method to  $D_{\text{new}}$  (if required)
12:    Evaluate  $D_{\text{new}}$  using the fitness function
13:    Apply local optimization to  $D_{\text{new}}$  (if required)
14:    Form  $D_t$  from  $D_{t-1}$  and  $D_{\text{new}}$  using the replacement method
15:  end if
16:  Select  $D_t^S \subseteq D_t$  using the selection method
17:  Optionally weight individuals in  $D_t^S$  (selection weighting)
18:  Learn a probabilistic model  $M_t$  from  $D_t^S$  using the learning method
19:   $t \leftarrow t + 1$ 
20: until the stopping condition is satisfied
```

```
11 | stats, cache = eda.run(verbose=True)
```

3.2.1 Input Parameters

pop_size Population size (integer).

n_vars Number of variables (integer).

fitness_func Callable that maps a 1-D NumPy array (an individual) to a scalar (single objective) or a 1-D array (multi-objective). pateda maximizes fitness; for minimization, negate the function.

cardinality For discrete problems: a 1-D integer array of length **n_vars** giving the number of values each variable can take. For continuous problems: a 2-D array of shape (2, **n_vars**) where row 0 contains lower bounds and row 1 upper bounds.

components An `EDAComponents` object specifying all the EDA building blocks (see Section 6).

random_seed Integer seed passed to NumPy's `default_rng`. Pass `None` for non-reproducible runs.

selection_weighting How the selected individuals are weighted when passed to the learning method. Options: "uniform" (default, equivalent to classic EDA), "proportional" (fitness proportional), "boltzmann" (Boltzmann distribution with temperature `weighting_beta`).

3.2.2 Output: Statistics and Cache

`eda.run()` returns a pair (`stats`, `cache`).

The `Statistics` object provides:

- `stats.best_fitness`: list of best fitness per generation.

- `stats.mean_fitness`: list of mean fitness per generation.
- `stats.std_fitness`: list of standard deviation per generation.
- `stats.best_individual`: the best solution found overall.
- `stats.best_fitness_overall`: its fitness value.
- `stats.generation_found`: the generation at which it was found.

The Cache object stores per-generation data controlled by a `CacheConfig`:

- `cache.populations`: list of population arrays.
- `cache.fitness_values`: list of fitness arrays.
- `cache.models`: list of learned models.
- `cache.selected_populations`: list of selected subsets.

Caching is *opt-in*: by default `CacheConfig` stores nothing (to keep long runs light), and the data are recorded only for the fields explicitly enabled and passed to `run()`:

```

1 from pateda.core.components import CacheConfig
2 cc = CacheConfig(cache_populations=True, cache_fitness=True,
3                 cache_models=True, cache_statistics=True,
4                 cache_selections=True) # or CacheConfig.
5                                     from_list([1,1,1,1,1])
6 stats, cache = eda.run(cache_config=cc, verbose=False)

```

The same mechanism works for every EDA family: `cache.models` stores the model container actually learned, so a univariate/Markov-network run yields a list of `FactorizedModel` objects, an EBNA/BOA run a list of `BayesianNetworkModel` objects, a Gaussian run a list of `GaussianModel` objects, and so on. This is what the knowledge-extraction module (Section 9) consumes to analyze the search a posteriori.

3.3 The EDAComponents Dataclass

The `EDAComponents` dataclass (defined in `core/components.py`) specifies all building blocks of the EDA. All fields except `learning`, `sampling`, and `stop_condition` are optional.

```

1 from pateda.core.components import EDAComponents
2 from pateda.seeding.random_init import RandomInit
3 from pateda.selection.truncation import TruncationSelection
4 from pateda.learning.umdla import LearnUMDA
5 from pateda.sampling.fda import SampleFDA
6 from pateda.replacement.elitist import ElitistReplacement
7 from pateda.stop_conditions.max_generations import MaxGenerations
8
9 components = EDAComponents(
10     seeding = RandomInit(),
11     selection = TruncationSelection(ratio=0.5),
12     learning = LearnUMDA(alpha=1.0),
13     sampling = SampleFDA(n_samples=300),
14     replacement = ElitistReplacement(n_elite=1),
15     stop_condition = MaxGenerations(100),
16 )

```


3.4 Example: UMDA on the OneMax Problem

```
1 import numpy as np
2 from pateda.core.eda import EDA
3 from pateda.core.components import EDAComponents
4 from pateda.seeding.random_init import RandomInit
5 from pateda.selection.truncation import TruncationSelection
6 from pateda.learning.umd import LearnUMDA
7 from pateda.sampling.fda import SampleFDA
8 from pateda.replacement.elitist import ElitistReplacement
9 from pateda.stop_conditions.max_generations_or_optimum import (
10     MaxGenerationsOrOptimum)
11
12 n_vars = 50
13 pop_size = 300
14
15 def onemax(x):
16     return np.sum(x)
17
18 cardinality = np.full(n_vars, 2) # binary variables
19
20 components = EDAComponents(
21     seeding = RandomInit(),
22     selection = TruncationSelection(ratio=0.5),
23     learning = LearnUMDA(alpha=1.0),
24     sampling = SampleFDA(n_samples=pop_size),
25     replacement = ElitistReplacement(n_elite=1),
26     stop_condition = MaxGenerationsOrOptimum(
27         max_gen=200, optimal_fitness=float(n_vars)),
28 )
29
30 eda = EDA(pop_size, n_vars, onemax, cardinality,
31           components, random_seed=42)
32 stats, cache = eda.run()
33 print("Best fitness:", stats.best_fitness_overall)
34 print("Best solution:", stats.best_individual)
```

3.5 Example: Gaussian UMDA on the Sphere Function

```
1 import numpy as np
2 from pateda import GaussianUMDA
3
4 def sphere(x):
5     return -np.sum(x**2) # maximise the negated sphere
6
7 alg = GaussianUMDA(
8     n_vars = 30,
9     bounds = (-5.0, 5.0),
10    fitness_func= sphere,
11    pop_size = 500,
12    n_gen = 100,
13    random_seed = 0,
14 )
15 stats, _ = alg.run()
16 print(f"Best: {stats.best_fitness_overall:.6f}")
```

3.6 Fitness Convention

pateda **maximizes** fitness. When the objective is to be minimized (e.g., energy, error, distance), the user should negate the function:

```
1 def minimization_objective(x):  
2     return -cost_function(x)    # pateda maximises this
```

4 Plug-and-Play Algorithm Classes

For convenience, pateda provides ready-to-use wrappers that assemble the appropriate components internally. These classes expose a minimal constructor and a `run()` method that returns `(stats, cache)`.

4.1 Discrete Algorithm Classes

All discrete algorithm classes share the following common parameters:

Parameter	Description
<code>n_vars</code>	Number of variables
<code>cardinality</code>	<code>int</code> (same for all vars) or 1-D integer array
<code>fitness_func</code>	Callable; higher is better
<code>pop_size</code>	Population size (default: 100)
<code>n_gen</code>	Number of generations (default: 50)
<code>selection_ratio</code>	Truncation fraction (default: 0.5)
<code>elitism</code>	Preserve best individual (default: True)
<code>random_seed</code>	RNG seed (default: None)
<code>alpha</code>	Laplace smoothing pseudo-count for probability tables (default: 1.0)

Unlike earlier EDA suites, the discrete algorithm classes apply *Laplace smoothing by default* (`alpha = 1.0`, i.e. add-one pseudo-counts). Because the samples available to a learner inside an EDA are small and biased toward the current best region, unsmoothed maximum-likelihood estimates routinely produce hard 0/1 marginals that prematurely freeze variables and collapse diversity. Add-one smoothing keeps every value reachable while barely perturbing well-supported estimates; users who want pure maximum-likelihood tables can still pass `alpha=0.0`.

Table 1 summarizes the discrete algorithm classes. While some important components are pre-defined in these plug-and-play algorithm classes (e.g., truncation selection), other components can be specified and different classes of plug-and-play algorithms can be easily defined due to the modular design.

4.2 Continuous Algorithm Classes

Table 2 summarizes the continuous algorithm classes.

Common parameters for continuous classes are `n_vars`, `bounds` (a tuple `(lo, hi)` or a `(2, n_vars)` array), `fitness_func`, `pop_size`, `n_gen`, `selection_ratio`, `elitism`, and `random_seed`.

4.3 Quick Comparison Example

```

1 from pateda import UMDA, TreeEDA, EBNA
2 import numpy as np
3
4 def deceptive3(x):
5     """Sum of order-3 deceptive functions."""
6     n = len(x)
7     score = 0.0
8     for i in range(0, n - 2, 3):
9         u = int(x[i]) + int(x[i+1]) + int(x[i+2])
10        score += 0.9 if u < 3 else 1.0
11    return score
12
13 n_vars = 30
14 for AlgClass in [UMDA, TreeEDA, EBNA]:
15     alg = AlgClass(n_vars=n_vars, cardinality=2,
16                   fitness_func=deceptive3,
17                   pop_size=500, n_gen=100, random_seed=1)
18     stats, _ = alg.run(verbose=False)
19     print(f"{AlgClass.__name__:10s} best={stats.best_fitness_overall
20         :.3f}")

```

5 Probabilistic Models and Algorithm Families

5.1 Discrete Representation

Discrete variables take values in $\{0, 1, \dots, r_i - 1\}$ where r_i is the cardinality of variable X_i . Binary variables have $r_i = 2$. `pateda` represents a population of N solutions as a 2-D NumPy array of shape (N, n) with integer dtype. The `cardinality` parameter is a 1-D integer array $[r_1, r_2, \dots, r_n]$. Mixed-cardinality problems (some variables binary, others multi-valued) are fully supported.

5.2 Continuous Representation

Continuous variables are real-valued, bounded by $[\ell_i, u_i]$. The population is a 2-D float array of shape (N, n) . The `cardinality` array for continuous problems is a 2-D array of shape $(2, n)$: row 0 contains the lower bounds and row 1 the upper bounds.

5.3 Factorized Distributions

A factorized distribution [55, 29] represents a joint probability over $\mathbf{X} = (X_1, \dots, X_n)$ as a product of factor (clique) marginals:

$$p(\mathbf{x}) = \prod_{i=1}^m p(x_{C_i} \mid x_{S_i}) \quad (1)$$

where C_i is the set of “new” variables in factor i and S_i is the set of “separator” variables shared with previous factors. This class of models, called *ordered factorizations* [67], subsumes:

- Marginal product factorizations (univariate, bivariate, etc.) [56, 29]
- Junction-tree factorizations [55, 59]
- Junction-graph factorizations [67]
- Markov chain factorizations [73]

pateda stores factorizations in the **FactorizedModel** container, which holds a list of factors; each factor records its clique variables, separator variables, and a conditional probability table. The **LearnFDA** class fits the tables from a given population, and **SampleFDA** samples new solutions using probabilistic logic sampling (PLS) [56].

Concretely, the container keeps two parallel lists (Figure 1). The **structure** array is the *cliques matrix*: each row encodes one factor as $[|S_i|, |C_i|, \text{separator indices } \dots, \text{new-variable indices } \dots]$, i.e. the separator variables S_i it conditions on and the new variables C_i it introduces. The **parameters** list stores, in the same order, the conditional probability table $p(x_{C_i} | x_{S_i})$ of each factor. Reading the rows in order gives an ancestral ordering, which is exactly what PLS needs: variables are instantiated factor by factor, each conditioned on already-sampled separators. This single representation is what lets one storage format and one sampler (**SampleFDA**) serve univariate models, trees, junction-tree and junction-graph factorizations, and Markov-chain factorizations alike — only the rows of the cliques matrix change.

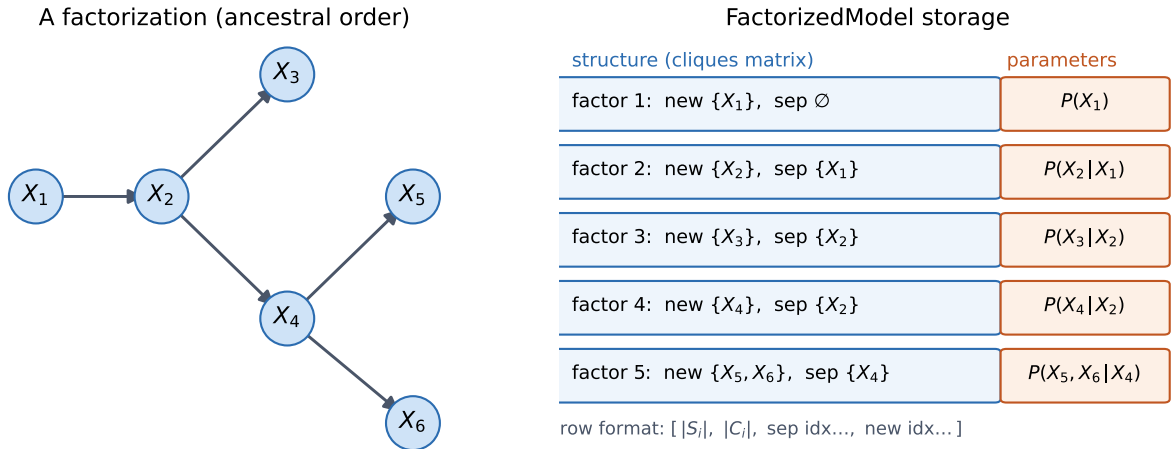


Figure 1: How **pateda** stores an arbitrary ordered factorization. A factorization over the variables (left) is kept as a cliques matrix (**structure**, one row per factor giving its separator and new variables) together with an aligned list of conditional probability tables (**parameters**).

FDA (**FDA** class): when a user supplies a fixed clique structure (known from the problem domain), **LearnFDA** computes only the conditional probability tables from data, bypassing structure learning entirely. This is the original factorized distribution algorithm of (**author?**) [55] where the structure of the interactions is not learned from data.

5.4 Univariate Models

Univariate models assume all variables are independent: $p(\mathbf{x}) = \prod_i p(x_i)$. The marginal $p(x_i)$ is estimated as the empirical frequency in the selected population (with optional Laplace smoothing α).

UMDA [56]. Univariate marginal distribution algorithm. Each marginal is learned independently from the selected population. Learning: **LearnUMDA**. Sampling: **SampleFDA**.

PBIL [3]. Population-based incremental learning. Maintains a running estimate of the marginals updated with learning rate α : $P^{(t+1)}(x_i) = (1 - \alpha) P^{(t)}(x_i) + \alpha \hat{P}(x_i)$. With $\alpha = 1$ PBIL reduces to UMDA. Learning: **LearnPBIL**. Sampling: **SampleFDA**.

MIMIC [14]. Mutual Information Maximization for Input Clustering. Learns a chain-structured factorization that maximizes the sum of pairwise mutual information along the chain. Learning: **LearnMIMIC**. Sampling: **SampleFDA**.

5.5 Tree and Forest Models

Tree-structured models represent pairwise variable dependencies using a maximum-weight spanning tree of the mutual-information matrix [12].

TreeEDA / Tree-EDA [5]. Learns a directed tree using Chow–Liu algorithm on pairwise MI. Learning: `LearnTreeModel`. Sampling: `SampleFDA`.

TreeEDAR / Tree-EDA_r [4, 72] Variant of Tree-EDA with a user-supplied `interaction_matrix` that constrains which pairs of variables can form a dependency. Learning: `LearnTreeModelR`.

TreeEDAM / Tree-EDA-M [70]. Variant of Tree-EDA that keeps only *malign* interactions. For each pair of variables it compares the most probable joint configuration $\arg \max_{a,b} p(x_i = a, x_j = b)$ with the configuration that maximizes the product of the univariate marginals ($\arg \max_a p(x_i = a), \arg \max_b p(x_j = b)$). If they coincide the interaction is *benign* — it only reinforces the main effects and can be reproduced by independent sampling — so its mutual information is not computed and no edge is added. Otherwise the interaction is *malign* (deceptive) and is treated as in Tree-EDA. The model is thus usually a forest, and reduces to UMDA when every interaction is benign. The test is defined through $\arg \max$ of the marginal tables, so it applies unchanged to discrete non-binary variables. Learning: `LearnTreeModelM`. Sampling: `SampleFDA`.

Int_FDA / Int-Tree [81]. Tree-based FDA for integer problems with *very high cardinality*. It uses the same Chow–Liu tree as Tree-EDA, but its model is *not* a probability distribution: rather than storing a conditional table $P(X_j | X_i)$ of size $c \times c$ per edge, it stores the tree together with two index tables and the selected population itself. Column i of `PopulValues` lists the indices of the selected vectors ordered by the value of X_i , and `ParentIndices` records, for each value, the boundaries of the corresponding block, so that “pick a selected vector whose parent equals v ” is an $O(1)$ operation. Sampling copies genes from donor vectors along the tree: the root is copied from a random selected vector, and each child X_j is copied from a random selected vector whose parent X_i matches the value already assigned. This reproduces the empirical tree distribution $p(x) = p(x_{root}) \prod_i p(x_i | x_{pa(i)})$ *exactly*, but needs only $O(N \cdot n + c \cdot n)$ memory instead of the $O(c^2 \cdot n)$ of a table-based tree, which is what makes it practical when c is large. Learning: `LearnIntFDA`. Sampling: `SampleIntFDA`.

BMDA [63] Bivariate marginal distribution algorithm. Detects pairwise dependencies with chi-square tests and builds a forest of bivariate conditionals. Learning: `LearnBMDA`.

AffEDA [75]. Affinity-propagation EDA Uses affinity propagation on the MI matrix to discover variable cliques, then builds a factorized model over those cliques. Learning: `LearnAffinityFactorization`.

MTED / MT-EDA [80]. Mixture of trees EDA. Learns k tree components with mixture weights. Weight learning options: "uniform", "em", or "fitness_proportional". Adaptive and prior-based extensions are supported to prevent premature convergence. Learning: `LearnMixtureTrees`. Sampling: `SampleMixtureTrees`.

5.6 Bayesian Network Models

Bayesian networks [44] are directed acyclic graphs in which X_i is conditionally independent of its non-descendants given its parents $\mathbf{Pa}_i$. The joint distribution factorizes as:

$$p(\mathbf{x}) = \prod_{i=1}^n p(x_i | \mathbf{pa}_i). \quad (2)$$

The local distribution $p(x_i \mid \mathbf{pa}_i)$ is a conditional probability table (CPT) with r_i entries for each of the $q_i = \prod_{X_j \in \mathbf{Pa}_i} r_j$ parent configurations.

EBNA [23]. Estimation of Bayesian network algorithm. Structure learning uses a greedy score-and-search procedure with the Bayesian information criterion (BIC) [82] or the Bayesian score [13]. Learning: `LearnEBNA`. Sampling: `SampleBayesianNetwork`.

BOA [62, 60]. Bayesian optimization algorithm. Greedy structure search with BIC/MDL scoring. Learning: `LearnBOA`. Sampling: `SampleBayesianNetwork`.

hBOA [61, 60]. Hierarchical Bayesian optimization algorithm. hBOA differs from BOA in the representation of the local conditional distributions: where BOA stores a full CPT with $q_i = \prod_{X_j \in \mathbf{Pa}_i} r_j$ rows, hBOA stores a decision tree or decision graph over the parents, so that only the parent configurations the data actually distinguishes receive their own parameters. The number of parameters then grows with the complexity of the dependency rather than with $|\mathbf{Pa}_i|$, allowing hBOA to admit substantially larger parent sets than BOA at the same population size. Decision graphs additionally *merge* leaves, letting distinct parent configurations share parameters, which is what makes hBOA effective on hierarchically decomposable problems. The complete algorithm of (author?) [60] also replaces generational replacement by restricted tournament replacement (RTR); as a replacement component this is orthogonal to the learning method and is configured separately in `EDAComponents`. Learning: `LearnHBOA`. Sampling: `SampleBayesianNetwork`.

LFDA [54]. Learning factorized distribution algorithm. Whereas FDA is given a factorization derived from the structure of an additively decomposable function, LFDA learns a Bayesian network from the selected population at each generation and uses it as the factorization. Structure search is greedy hill-climbing on a BIC score whose complexity penalty carries a weighting factor α , $\text{BIC}(B, D, \alpha) = \sum_i \log P(D_i \mid \mathbf{Pa}_i, \theta_{ML}) - \alpha \frac{\log N}{2} |\theta|$, subject to a hard bound k on the number of parents per variable. Increasing α yields sparser networks and is the principal control on overfitting given the small samples available inside an EDA. Learning: `LearnLFDA`. Sampling: `SampleBayesianNetwork`.

PADA [88, 58]. Polytree approximation of distribution algorithm. PADA restricts the model to a *polytree*, a singly connected Bayesian network whose undirected skeleton contains no loops. Unlike the score-and-search algorithms above, PADA learns the structure with independence tests: the LPA algorithm builds a candidate edge list from marginal dependencies $\text{Dep}(a, b) > e_0$, discards pairs explained away by a third variable ($\text{Dep}(a, b \mid c) < e_1$), ranks the survivors by the global dependency degree, inserts edges while keeping the skeleton singly connected, and orients head-to-head nodes using the fact that conditioning on a collider *raises* the dependency between its parents. A polytree has at most $n - 1$ edges, so its parameters remain estimable from small populations, while still admitting multiple parents and colliders that a tree model cannot express. Learning: `LearnPADA`. Sampling: `SampleBayesianNetwork`.

BN-based EDAs with alternative structure learners. EBNA, BOA and LFDA are close relatives: all three learn the network by greedy score-and-search. The following plug-and-play classes keep the same plug-and-play interface and the same `SampleBayesianNetwork` sampler but replace the structure learner by one from a *different* paradigm. They are motivated by the observation that a score-and-search learner already covers the strongest online performers, so these methods earn their place through scalability and by excelling on particular problem classes.

SARTRE-EDA [?]. Learns the network with SARTRE, an order-based method that prunes spurious edges through a sparse additive model fitted by group-lasso regression. It is

very fast, scales to large n , and produces the most accurate skeletons (F_1) among the compared learners, making it the safest of the alternative-paradigm methods. Learning: `LearnSARTRE`. Sampling: `SampleBayesianNetwork`.

BINOTEARS-EDA [?]. Learns the network with BINOTEARS, which casts binary-data structure learning as a differentiable continuous optimization with an acyclicity constraint. It scales via gradient optimization and recovers the additive building blocks of decomposable problems (Trap, Deceptive) accurately. *Binary variables only*. Learning: `LearnBINOTEARS`. Sampling: `SampleBayesianNetwork`.

PCBN-EDA [? ?]. Constraint-based learning with the order-independent PC-Stable algorithm in a *bounded conditioning-order* form: the size of the conditioning set in the independence tests is capped at k (`max_cond_set_size`), which caps the number of tests and is what lets the constraint-based paradigm scale to $n \geq 100$. It is competitive on dependency-dense, lattice-structured problems such as Ising. Learning: `LearnPCBN`. Sampling: `SampleBayesianNetwork`.

HSARTRE-EDA / HBINOTEARS-EDA . The hBOA-style upgrade of the two scalable learners above. Following (author?) [60], the base method (SARTRE or BINOTEARS) discovers the skeleton, a decision graph then learns the local, context-specific structure *restricted to that skeleton*, and generational replacement is replaced by restricted tournament replacement (RTR) so that alternative partial solutions survive until their context is resolved. The decision graph lets the local conditional distributions grow with the complexity of the dependency rather than with $|\mathbf{Pa}_i|$, and the niching preserves the diversity that decomposable and multi-modal problems require. These are the only classes in this group that pair a niching replacement with the learner by default. Learning: `LearnHSARTRE` / `LearnHBINOTEARS`. Replacement: `RestrictedTournamentReplacement`. Sampling: `SampleBayesianNetwork`.

5.7 Most Probable Configurations

Both EBNA and BOA sample new individuals using probabilistic logic sampling over the directed network. Beyond random sampling, `pateda` can also generate the *deterministic* high-probability configurations of a learned model. This is motivated by the observation that, especially in late generations, the mode of the model is often a very good (sometimes the optimal) solution, and that inspecting several modes reveals the multiple basins the model has captured [21].

The single *most probable explanation* (MPE) is the configuration $\mathbf{x}^* = \arg \max_{\mathbf{x}} p(\mathbf{x})$. More generally, the k *most probable configurations* (k -MPC / k -MAP) are the k highest-probability configurations, which `pateda` computes exactly, per model type, rather than by enumeration:

Univariate models `KMPCUnivariate` uses Nilsson’s partition scheme: the top configuration is the per-variable mode, and the next best ones are obtained by the cheapest single-variable substitutions.

Tree / junction-tree / factorized models `KMPCTree` and `KMPCJunctionTree` run max-product (max-sum in log space) message passing followed by Nilsson’s most-probable-flip backtrack-ing to enumerate the ordered list of configurations.

Bayesian networks `KMPCBayesianNetwork` performs the same k -MAP over the CPD factorization of an EBNA/BOA network.

Markov networks the engine in `inference/map_inference.py` provides the single MPE by exact junction-tree inference, max-product belief propagation, decimation (iteratively fixing the most certain variable), or a greedy fallback.

The dispatcher `compute_kmpc(model, cardinality, k)` selects the right algorithm automatically from the model type and returns a `KMPCResult` with the configurations and their (log-)probabilities; `compute_map` returns just the MPE. These can be used as a sampler that injects the model’s modes into the new population, or as an analysis tool after a run.

5.8 Markov Network Models

A Markov random field (MRF) [27] is an undirected graphical model in which the joint distribution factorizes over the cliques of the graph as a Gibbs distribution:

$$p(\mathbf{x}) = \frac{1}{Z} \exp(-H(\mathbf{x})), \quad H(\mathbf{x}) = \sum_{C \in \mathcal{C}} \Phi_C(\mathbf{x}_C) \quad (3)$$

where $\mathcal{C}$ is the set of maximal cliques and Z is the partition function. Sampling from MRFs typically uses Gibbs sampling: iteratively sample each variable X_i from its conditional distribution given the current values of its neighbors $N(X_i)$.

MN-FDA / MNFDA [66]. Detects pairwise dependencies with a χ^2 independence test, extracts the maximal cliques of the resulting dependency graph, and turns them into an ordered factorized model (junction graph). The model can be sampled with PLS (`SampleFDA`, recommended for small cliques) or Gibbs sampling. Learning: `LearnMNFDA`. Sampling: `SampleFDA`.

MN-FDA_r / MNFDAR [67]. MN-FDA *restricted* to a known interaction structure: dependency testing is performed only for the variable pairs allowed by a binary interaction matrix R , so the learned network is a subgraph of the unrestricted MN-FDA network consistent with the a-priori problem structure (Section 10). Learning: `LearnMNFDAr`. Sampling: `SampleFDA`.

MN-FDAg / MNFDAG [67]. Identical pipeline to MN-FDA but using the G -test of independence ($G(X_i, X_j) = 2N \cdot \text{MI}(X_i, X_j)$, with $\text{df} = (r_i - 1)(r_j - 1)$) instead of the χ^2 threshold. The G -test is a more principled likelihood-ratio criterion and behaves better for variables of differing cardinality. Learning: `LearnMNFDAg`. Sampling: `SampleFDA` or `SampleGibbs`.

MN-FDAg_r / MNFDAGR [67]. MN-FDAg *restricted* to a known interaction matrix R , the G -test counterpart of MN-FDA_r. Learning: `LearnMNFDAgr`. Sampling: `SampleFDA` or `SampleGibbs`.

MOA [84, 85]. Markovianity-based optimization algorithm. Learns local k -neighborhoods for each variable and samples via Gibbs sampling. Learning: `LearnMOA`. Sampling: `SampleGibbs`.

The four MN-FDA variants therefore differ along only *two* orthogonal axes, and are best understood as a 2×2 table rather than four unrelated methods: the *dependency test* used to decide which edges enter the Markov network (χ^2 for MN-FDA/MN-FDA_r vs. the G -test for MN-FDAg/MN-FDAg_r), and whether structure learning is *unrestricted* (MN-FDA, MN-FDAg) or *restricted* to an a-priori interaction matrix R (the “_r” variants). The downstream representation (an ordered factorized/junction-graph model) and the available samplers (PLS or Gibbs) are shared by all four.

5.9 Markov Chain Models

MK-EDA / MKEDA [69]. k -order Markov chain EDA. Each variable X_i depends on the k preceding variables in a fixed linear ordering: $p(\mathbf{x}) = p(x_1) \prod_{i=2}^n p(x_i | x_{i-k}, \dots, x_{i-1})$. Well-suited to problems with a natural sequential structure (e.g. protein folding with relative encodings). Learning: `LearnMarkovChain`. Sampling: `SampleMarkovChain`.

MkRg-EDA [68]. Regularized k -order Markov EDA. It keeps the k -order Markov structure of MK-EDA but replaces each conditional probability table — whose size is exponential in k — by a *regularized multinomial logistic regression* that predicts X_i from (a function of) its previous k variables, fitted with the elastic net [94]. This keeps the number of parameters polynomial in k , so larger orders become affordable, and the regularization prunes the previous variables that do not contribute. Three predictor variants of increasing complexity are available through the `variant` argument: `"rgk"` (the k previous variables, $O(k)$ parameters), `"bivrgk"` (their pairwise products, $O(k^2)$) and `"allrgk"` (both, $O(k^2)$). Sampling generates the variables in order, seeding the first k from the selected population and drawing each remaining variable from its regressed conditional distribution. Learning: `LearnRegularizedMarkov`. Sampling: `SampleRegularizedMarkov`.

5.10 Constrained EDAs

CUMDA [77]. Unitation-constrained UMDA. Every solution must contain exactly `n_ones` ones. The seeding method (`SeedingUnitationConstraint`) generates feasible initial solutions, and the sampling method (`SampleCUMDA`) uses Stochastic Universal Sampling to maintain the constraint. Learning: `LearnCUMDA`.

CFDA [78]. Unitation-constrained FDA. Uses a tree-structured factorized model while enforcing the same fixed-union constraint as CUMDA. Learning: `LearnCFDA`. Sampling: `SampleCFDA`.

5.11 Continuous Gaussian Models

While `pateda` includes several continuous-based EDAs, these methods have not been tested, enhanced, and extended to the same extent as the discrete implementations. Other Python libraries implement more comprehensive approaches to the optimization of continuous problems with model-based evolutionary algorithms: the CMA-ES family [57] and, closest in spirit to `pateda`, the `EDAspy` package [87], which provides a range of univariate, multivariate-Gaussian and copula-based continuous EDAs. Users targeting continuous optimization are encouraged to consider these libraries alongside `pateda`.

GaussianUMDA [42, 41]. Each variable is modeled independently as a Gaussian: $p(x_i) = \mathcal{N}(x_i; \mu_i, \sigma_i^2)$. Parameters (μ_i, σ_i^2) are estimated from the selected population. Learning: `LearnGaussianUnivariate`. Sampling: `SampleGaussianUnivariate`.

GaussianFull [41, 8]. Full multivariate Gaussian: $p(\mathbf{x}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma)$. All pairwise dependencies are captured by the covariance matrix Σ . May suffer from variance collapse in later generations[6]; a variance scaling parameter is provided. Learning: `LearnGaussianFull`. Sampling: `SampleGaussianFull`.

GaussianNetworkEDA Gaussian network (GMRF) EDA. Learns a sparse Gaussian graphical model where each variable's local distribution is a linear regression on its parents in a directed graph. The GMRF approach identifies relevant conditional independences in continuous data.

MixtureGaussianEDA [10]. Mixture of Gaussian distributions. The population is first clustered (k-means or similar), then a separate Gaussian (univariate or full) is fit to each cluster. Suitable for multimodal search landscapes. Learning: `LearnMixtureGaussian`. Sampling: `SampleMixtureGaussian`.

5.12 Learning from Weighted Solutions

All the learners above estimate their model from the selected set by treating every selected solution equally. `pateda` generalizes this by letting each selected individual carry a *weight* that scales its contribution to the sufficient statistics (counts, mutual information, covariances) of *any* model. This blends structure/parameter learning with fitness guidance without changing the learner: a weight of 1 for every individual recovers the classic EDA, while fitness-dependent weights let better solutions influence the model more. The behaviour is controlled by the `selection_weighting` argument of the EDA constructor:

```
1 eda = EDA(pop_size, n_vars, fitness_func, cardinality,
2           components,
3           selection_weighting="boltzmann",    # "uniform" / "
           proportional" / "boltzmann"
4           weighting_beta=2.0)    # higher beta -> stronger fitness
           pressure
```

"uniform" is the classic EDA; "proportional" weights each selected solution by its (normalized) fitness; "boltzmann" weights it by $e^{\beta f(\mathbf{x})}$. This subsumes as a special case the older fitness-weighted univariate scheme (the "Bisection"/BSC method of (author?) [35]), which is exactly a univariate learner run with proportional weighting; for this reason BSC is no longer provided as a separate algorithm class. Because the weighting acts at the sufficient-statistic level it composes with every model family — univariate, tree, Bayesian network, Markov network, factorized, and Gaussian alike.

6 EDA Components

6.1 Seeding Methods

Seeding methods initialize the population at generation 0. In an EDA the seed matters more than in a genetic algorithm: it is the only sample the very first model is estimated from, so a biased or infeasible seed directly biases the first learned distribution. Seeding is therefore also the natural entry point for a-priori knowledge (biased marginals, feasibility constraints, or a warm-start population from a previous run; Sections 10 and 9.5).

RandomInit Samples each variable uniformly from its range. For discrete variables: integer in $[0, r_i)$. For continuous variables: real in $[\ell_i, u_i]$.

BiasInit Biased binary initialization. Variable X_i is set to 1 with probability p and to 0 with probability $1 - p$.

ProbabilisticInit Generalizes **BiasInit** to arbitrary discrete variables: each variable is sampled from a user-supplied categorical distribution over its `cardinality` values, so the initial population follows a chosen product distribution $\prod_i P(X_i)$. It is the natural way to seed a non-binary problem with per-configuration prior beliefs (a form of a-priori knowledge; **BiasInit** is the binary special case).

SeedingUnitationConstraint Generates binary solutions with exactly `num_ones` ones. Required by **CUMDA** and **CFDA**.

SeedThisPop Uses a user-supplied population as generation 0 instead of generating one. The population is passed at run time through the `initial_population` parameter (an array of shape `(pop_size, n_vars)`); the method validates its shape and returns it unchanged. This is the mechanism used to warm-start a run from the output of a previous EDA.

6.2 Selection Methods

Selection determines which individuals are used for model learning. This is the single most consequential component of an EDA: it defines the empirical distribution the model is fitted to, and hence the search bias. Whereas in a genetic algorithm selection only decides who reproduces, in an EDA it decides *what is modelled* — stronger selection sharpens the target distribution but risks collapsing diversity and inducing spurious dependencies from a small, skewed selected set. The role of selection in EDAs has been studied by (author?) [36, 46, 50, 71].

TruncationSelection Selects the top $\lfloor \text{ratio} \cdot N \rfloor$ individuals by fitness. This is the most common selection method in EDAs.

TournamentSelection Tournament selection of configurable size.

ProportionalSelection / **SUSSelection** Fitness-proportional selection using stochastic universal sampling (SUS) [2].

BoltzmannSelection Selection with Boltzmann temperature; probability proportional to $e^{\beta f(x)}$.

RankingSelection Rank-based selection; probability proportional to $2r/(N(N+1))$ where r is the fitness rank.

NonDominatedSelection Selects the non-dominated front. For multi-objective problems.

CrowdingSelection NSGA-II-style: non-dominated sorting + crowding distance tie-breaking.

IndicatorBasedSelection SMS-EMOA-style hypervolume indicator-based selection.

ParetoFrontSelection Selects complete Pareto fronts up to the selection quota.

6.3 Replacement Methods

Replacement decides which solutions survive into the next generation and thus controls the population’s turnover. In EDAs the default is full generational replacement, because each generation is meant to be a fresh sample from the current model; elitism is nonetheless useful to avoid losing the best solution when the model drifts. Replacement is also where *diversity-preserving* schemes (crowding, restricted tournament, clustering) belong — they let an EDA maintain several basins at once, which is important on multimodal problems and is the mechanism the complete hBOA relies on (Section 5.6).

ElitistReplacement Copies the best `n_elite` solutions from the previous generation into the new population if they are better than the worst new individual. When `n_elite` is left unset the number of elites defaults to `elite_ratio = 0.1` of the population; the plug-and-play algorithm classes instead instantiate it with `n_elite = 1` (preserve the single best) when `elitism=True`.

GenerationalReplacement New population completely replaces the old one.

DeterministicCrowdingReplacement [49]. Deterministic crowding: each new individual competes with the *most similar* current individual (Hamming distance for discrete, Euclidean for continuous) and replaces it only if at least as fit, so distinct niches survive.

RestrictedTournamentReplacement [30]. Restricted tournament replacement (RTR): each new individual competes with the most similar member of a random window of size `window_size`; the complete hBOA (Section 5.6) relies on this scheme.

ClusteringReplacement . The merged population is clustered (k-means) and elitism is applied *within* each cluster, with a per-cluster quota proportional to the cluster size, preserving one niche per cluster.

The three diversity-preserving schemes share the **ReplacementMethod** interface, so they can be dropped into any pipeline and are useful on multimodal problems where plain elitism would collapse the population onto one basin.

6.4 Stopping Conditions

A stopping condition ends the generational loop. Besides the usual budget and target-value criteria, EDAs admit a natural model-based stopping signal: once the learned distribution has converged (marginals near 0/1, population diversity near zero) further sampling cannot escape the current basin, so detecting convergence avoids wasting evaluations.

MaxGenerations Stops after a fixed number of generations.

MaxGenerationsOrOptimum Stops when either the generation limit is reached or a known optimal fitness value (passed as `optimal_fitness`) is attained.

NoImprovement Stagnation test: stops when the best fitness has not improved by more than `epsilon` over the last `k` generations.

PopulationConvergence Stops when the population diversity drops below `tol` (mean per-variable value agreement for discrete populations, relative dispersion for continuous ones), optionally requiring the condition to persist for a `patience` of generations.

CompositeStop Combines several conditions with an `"any"` (stop when the first fires) or `"all"` rule, e.g. a hard generation cap together with a stagnation or convergence test.

6.5 Local Optimization

Local optimization turns an EDA into a *memetic* algorithm: sampled solutions are refined by a local search before selection. This is especially effective for EDAs, because polishing solutions sharpens the statistical signal the model learns from — building blocks are completed, so the dependencies between their variables become easier to detect — while the global model supplies the diversification the local search lacks. The two operators of Section 6.6 go one step further and localize the search using the model's own structure.

GreedySearch Greedy hill climbing in the continuous case.

DiscreteGreedySearch Greedy flip-based local search for discrete problems.

DiscreteSimulatedAnnealing Simulated annealing for discrete problems.

ContiguousBlockOptimization Specialized local search for contiguous-block problems.

ScipyLocalSearch Wraps `scipy.optimize.minimize` for continuous local optimization. Any `scipy` method can be selected (e.g. `"L-BFGS-B"`, `"Nelder-Mead"`, `"Powell"`, `"CG"`), with a per-solution iteration and function-evaluation cap.

For continuous problems, combining a Gaussian EDA with different `scipy` local searches is a one-line change of the `method` argument:

```

1 from pateda.local_optimization.scipy_local_search import
   ScipyLocalSearch
2 # gradient-based refinement of the sampled solutions
3 local = ScipyLocalSearch(method="L-BFGS-B", max_iter=100,
   max_fun_evals=200)
4 # ... pass local as EDAComponents(local_optimization=local, ...)

```

A worked comparison of several `scipy` methods inside a Gaussian EDA on Rastrigin is provided in `examples/continuous_memetic_eda.py`, where Powell’s method drives the EDA to the global optimum that the plain EDA does not reach.

6.5.1 Budget- and subset-aware memetic local search

A family of discrete local optimizers designed to be used as the local-search step of a memetic EDA: after sampling, they are applied to a *subset* of the population under a shared *evaluation budget*. All of them share one interface — the base class `BudgetedLocalSearch` — so they are fully interchangeable in a pipeline. Two parameters, common to every optimizer, control the local-search intensity: `subset_fraction` (the fraction of sampled solutions the search is applied to, selected as the best, a random subset, or the worst) and `evaluation_budget` (the total number of fitness evaluations spent per generation, shared equally among the selected solutions). A neighbor changes one variable to another value of its domain, so the optimizers work for binary and non-binary discrete variables alike.

DeterministicHillClimber [65]. Steepest-ascent single-change hill climber (the deterministic hill climber, DHC): the whole single-change neighborhood is evaluated and the best-improving move applied, until a local optimum is reached.

FirstImprovementHillClimber [28]. First-improvement descent: neighbors are scanned in random order and the first improving move is applied.

StochasticHillClimber . Random-mutation hill climbing (RMHC): a single random change is proposed each step and accepted iff it does not worsen the fitness.

SimulatedAnnealing [38]. Metropolis simulated annealing over the single-change neighborhood, with geometric or linear cooling stretched over the evaluation budget and an optional automatic initial temperature.

VariableNeighborhoodSearch [28]. Basic VNS: shake the incumbent in the growing k -th (Hamming distance k) neighborhood, run a local descent, and keep the result only if it improves, otherwise widen the shake.

ReducedVariableNeighborhoodSearch [28]. Reduced VNS: the same neighborhood-change scheme without the descent, for problems where evaluations are expensive.

6.6 Exploiting the model structure: network crossover and substructural search

Most samplers in `pateda` draw new solutions from the *probabilities* of the learned model (probabilistic logic sampling, Gibbs sampling, most-probable configurations). The graphical *structure* of the model can also be exploited directly, in two ways that do not use its probabilities at all. Both are generic: a learned model of any class (Bayesian network, tree, factorized or Markov-network model) is mapped to a graph view by `model_to_linkage_graph` / `model_to_substructures` in `knowledge_extraction`, and the very same operators also accept a linkage graph known a priori from the problem (an additive function’s blocks, an Ising lattice, a UBQP weight matrix, a SAT clause graph).

SampleNetworkCrossover [32]. Network crossover: a recombination operator (implemented as a **SamplingMethod**, with the population supplied as the parent pool). It grows a *connected* crossover mask on the linkage graph — by randomized breadth-first search ("**bfs**") or a random walk ("**random_walk**") — and swaps the masked variables between two parents. Because the mask is connected, its boundary cuts the *weakest* dependencies, so tightly-linked building blocks are inherited together and not disrupted. It is the structural analogue of the crossover operators in **pateda.crossover**, and the recombination counterpart of the partial samplers: partial sampling *resamples* a subset of variables from the model, network crossover *swaps* a connected subset between parents.

SubstructuralLocalSearch [47]. Substructural neighborhood search: a local search (a **LocalOptMethod** subclassing **BudgetedLocalSearch**, so it shares the **subset_fraction** / **evaluation_budget** intensity controls) that hill-climbs over the joint values of whole *substructures* rather than single variables. For a Bayesian network the substructure of X_i is $\{X_i\} \cup \mathbf{Pa}_i$ ("**parental**"), $\{X_i\} \cup \Omega_i$ ("**children**") or their union ("**both**"); for a factorized / Markov model it is a clique ("**clique**"); a generic Markov-blanket neighborhood ("**neighborhood**") is also available. Each substructure's $\prod_{i \in S} r_i$ value combinations are enumerated and the best-improving one is accepted using the actual fitness. Optimizing a linkage group as a unit escapes the deception that traps a single-variable hill climber, providing the intensification that complements the model-based global search. The EDA passes the current model to the local search automatically.

The two operators express the rationale of exploiting the learned structure in complementary ways: network crossover recombines while *avoiding model disruption*, and substructural search *intensifies* the search within the model's linkage groups.

6.7 Crossover and Mutation as Learning and Sampling

A useful consequence of the learning/sampling decomposition at the heart of **pateda** is that classical genetic operators fit the same interface: a recombination or mutation operator can be expressed as a (non-probabilistic) *learning* step that records how offspring will be produced, followed by a *sampling* step that produces them. The “model” is then simply the procedural description of the operator rather than a probability distribution. This is the *non-probabilistic model* paradigm of MATEDA-2.0, and it lets crossover and mutation be dropped into a pipeline, combined with model operators, and searched over by the grammar (Section 12) on an equal footing with UMDA, EBNA, and the rest.

Concretely, **pateda** implements the classic *two-point crossover* this way. Its learning phase (**LearnTwoPointCrossover**) pairs up the selected solutions and, for each pair, draws two cut points, storing the pairs and cut points in the model; its sampling phase (**SampleTwoPointCrossover**) applies those cuts to generate the offspring, optionally followed by a mutation operator. The same recipe yields **Learn/SampleBlockCrossover** (block/uniform-style exchange) and **Learn/SampleTransposition** (a permutation-style transposition operator); mutation operators such as **RandomResetMutation** plug into the **mutation** slot of **EDAComponents**. Any further crossover operator can be added by following the same two-phase split: store the procedure when learning, execute it when sampling.

6.8 Repairing Methods

BoundsRepair Clips continuous variable values to their feasible bounds.

TrigonometricRepair Trigonometric projection for angular variables.

UnitationRepair Adjusts binary solutions to satisfy a fixed unitation constraint.

7 Test Functions and Benchmark Problems

The benchmark suite is organized by *representation* and by *kind* inside the `functions/` package, so that the intended cardinality of a problem and whether it is an academic toy or a combinatorial problem are explicit from its location:

Group	Contents
<code>discrete_binary/toy_functions/</code>	Pseudo-Boolean academic benchmarks: OneMax, trap/deceptive, NK landscapes, contiguous-block functions.
<code>discrete_binary/problems/</code>	Binary combinatorial problems: Ising, UBQP, SAT, HP protein folding, max-clique and other graph problems.
<code>discrete_non_binary/toy_functions/</code>	Integer / categorical benchmark functions (multi-valued deceptive, etc.).
<code>discrete_non_binary/problems/</code>	Non-binary combinatorial problems (e.g. graph colouring, Braid).
<code>continuous/toy_functions/</code>	Real-valued benchmarks: Sphere, Rastrigin, Rosenbrock, Ackley, Griewank.
<code>continuous/problems/</code>	Continuous problems: AB off-lattice protein model, GNBG instances.

Shared readers for graph-instance files live in `functions/graph_utils.py` and serve both the binary and non-binary graph problems. Figure 2 illustrates the interaction structure that characterizes several of these problems — the very structure a good EDA is expected to recover in its model.

7.1 Discrete Functions

OneMax Maximizes $f(\mathbf{x}) = \sum_i x_i$. The simplest EDA benchmark; all variables are independent.

Trap functions Deceptive order- k subfunctions [1]: $f(\mathbf{x}) = \sum_j \text{trap}_k(u_j)$ where u_j is the number of ones in block j . Optimal solution is the all-ones string; the deceptive attractor is the all-zeros string.

Deceptive functions Goldberg’s deceptive functions [25].

Additively decomposable functions (ADFs) $f(\mathbf{x}) = \sum_i f_i(\mathbf{x}_{S_i})$ where each subfunction f_i is defined over a disjoint subset S_i of variables. Ideal for studying the structural identification capabilities of EDAs.

NK landscapes [37]. Each variable depends on k others drawn uniformly or in circular topology. The subfunction value for each configuration is drawn at random.

Ising model Energy minimization on a lattice: $E = -\sum_{i<j} J_{ij}\sigma_i\sigma_j - \sum_i h_i\sigma_i$.

HP protein folding [19]. 2-D/3-D lattice model. Solutions encode the sequence of relative moves of amino acid residues. pateda uses the relative encoding with a backtracking repair heuristic.

SAT [83]. Satisfiability problems; single- and multi-objective versions available.

UBQP Unconstrained binary quadratic programming: $f(\mathbf{x}) = \mathbf{x}^T Q \mathbf{x}$.

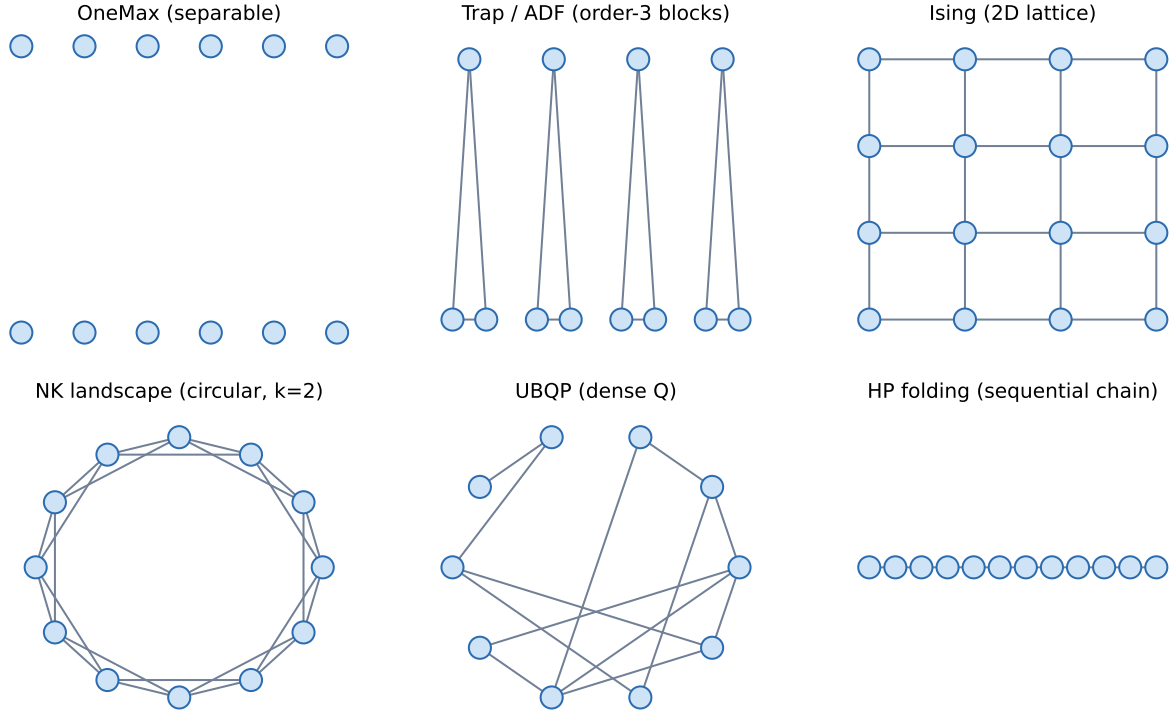


Figure 2: Variable-interaction structure of representative benchmark problems. Separable problems (OneMax) have no edges; additively decomposable / trap functions decompose into disjoint blocks; the Ising model is a lattice; NK landscapes use a (here circular) neighborhood; UBQP has a dense interaction matrix; and the HP folding model has a sequential chain structure. References: trap/deceptive [1, 25], NK [37], HP [19], SAT [83].

Contiguous-block functions Reward solutions with long contiguous blocks of identical bits.

Integer functions Functions over multi-valued (non-binary) discrete variables.

7.2 Continuous Functions

Standard benchmarks (`functions/continuous/benchmarks.py`): Sphere, Rastrigin, Rosenbrock, Ackley, Griewank.

AB off-lattice protein model [42]. Off-lattice protein model with Lennard-Jones interactions. Variables are bond angles in $[0, 2\pi]$.

GNBG instances [90] Generalized numerical benchmark generator instances (`GNBG_Instances.Python-main/`

`pateda` includes examples on how to test EDA implementations on discrete benchmarks such as the PBO suite [20] through linking with different Python libraries of the Iterative Optimization Heuristics ecosystem [89]. Examples on how to link to the Generalized Numerical Benchmark Generator (GNBG) of continuous functions[90] are also included.

8 Multi-Objective Optimization

8.1 Multi-Objective Problem Definition

A multi-objective optimization problem (MOP) [16] requires simultaneously optimizing m objectives $f_1(\mathbf{x}), \dots, f_m(\mathbf{x})$. Since objectives usually conflict, the goal is to find the *Pareto front*:

the set of non-dominated solutions where no single objective can be improved without degrading another.

While `pateda` is not a specialized library for solving multi-objective problems, it addresses these problems by modifying the selection scheme in such a way that different multi-objective approaches are indirectly implement.

For multi-objective problems, the objective function is supposed to return a vector of m values:

```

1 def bi_objective(x):
2     f1 = np.sum(x)                # first objective (maximise)
3     f2 = -np.sum((x - 0.5)**2)   # second objective (maximise)
4     return np.array([f1, f2])

```

8.2 Pareto Dominance

`pateda` implements Pareto dominance in `multiobjective/dominance.py`. Solution $\mathbf{x}$ dominates $\mathbf{y}$ if it is at least as good on all objectives and strictly better on at least one.

8.3 Multi-Objective Selection

Non-dominated sorting + crowding (NSGA-II) [17]. Solutions are ranked into Pareto fronts. Within each front, crowding distance breaks ties. Available via `CrowdingSelection`.

Indicator-based (SMS-EMOA) Hypervolume contribution per individual. Available via `IndicatorBasedSelection` and `multiobjective/indicators.py`.

Decomposition (MOEA/D) [93]. Decomposes the MOP into scalar subproblems via weight vectors. `pateda`'s MOEA/D is implemented in `multiobjective/moea.py`.

Model-based decomposition approaches to multi-objective optimization, which replace MOEA/D's genetic operators by probabilistic-model sampling, have been studied extensively by (author?) [91, 92] and are the conceptual template for the MO-EDAs assembled with `pateda`.

8.4 Metrics, Archive, and Offspring Generation

Quality indicators. `multiobjective/indicators.py` computes the standard set-quality metrics used to compare approximation fronts: the exact dominated *hypervolume* (any number of objectives), per-solution hypervolume *contributions* (used by the SMS-EMOA/IBEA-style selection), the *inverted generational distance* (IGD) to a reference front, and the *additive ϵ -indicator* matrix. These let a run be scored against a known Pareto front or against the union front of a set of algorithms.

External archive. An external `ParetoArchive` (`multiobjective/archive.py`) keeps the mutually non-dominated solutions discovered across generations, with an optional capacity bound (pruned by crowding). It decouples the reported approximation set from the working population, so elitist multi-objective behaviour is preserved even when the population itself is fully replaced each generation. MOEA/D maintains such an archive and records its size history.

Offspring generation in MOEA/D. MOEA/D associates each weight vector with a subproblem and a *neighborhood* of nearby weight vectors. New solutions for a subproblem are generated by mating within its neighborhood (and learning the model from the neighborhood's solutions), and a freshly generated solution updates every neighboring subproblem it improves under the scalarizing function (`multiobjective/scalarization.py`). This neighborhood-restricted generation is what gives decomposition approaches their characteristic spread of search effort along the front.

Tracking indicators during a run. The `MultiObjectiveTracker` (a `StatisticsMethod`) records the hypervolume — and, when a reference front is supplied, the IGD — of the current population every generation, storing them in `stats.custom["hypervolume"]` and `["igd"]`. It is attached like any statistics component:

```
1 from pateda.statistics import MultiObjectiveTracker
2 components = EDAComponents(..., statistics=MultiObjectiveTracker())
```

The example `multiobjective_variants_comparison.py` uses these tools to compare the three paradigms — NSGA-II-style dominance+crowding, SMS-EMOA indicator-based selection, and MOEA/D — on a shared bi-objective problem, scoring each approximation front with hypervolume and IGD (Figure 3).

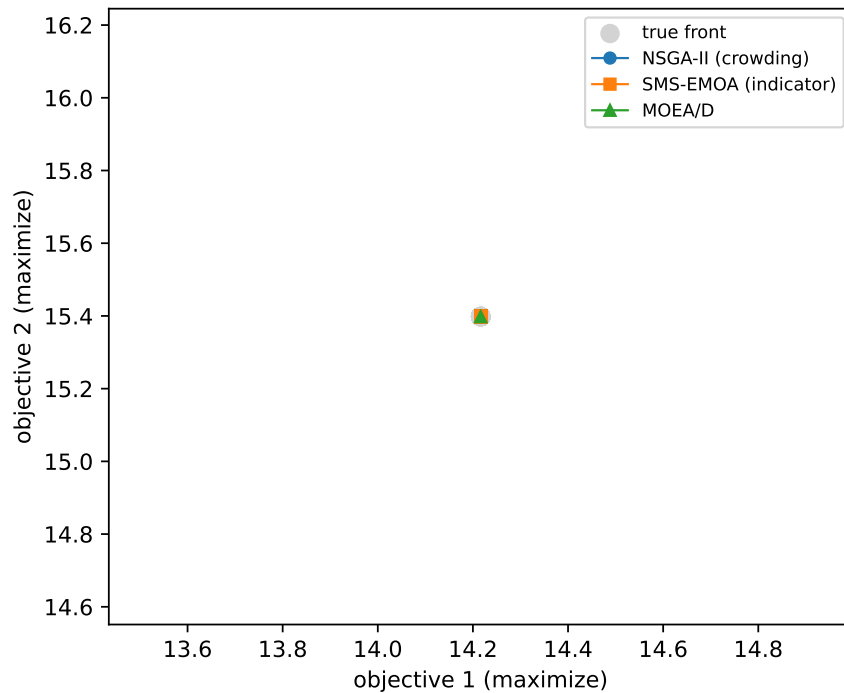


Figure 3: Approximation fronts obtained by the three multi-objective paradigms on a bi-objective linear 0/1 problem, against the true Pareto front. All three recover the front; hypervolume and IGD are reported by the accompanying example.

8.5 Multi-Objective EDA Pipeline

Below is a sketch of a multi-objective EDA using NSGA-II-style selection with a Bayesian network learner:

```
1 from pateda.core.eda import EDA
2 from pateda.core.components import EDAComponents
3 from pateda.seeding.random_init import RandomInit
4 from pateda.selection.crowding import CrowdingDistanceSelection
5 from pateda.learning.ebna import LearnEBNA
6 from pateda.sampling.bayesian_network import SampleBayesianNetwork
7 from pateda.replacement.elitist import ElitistReplacement
8 from pateda.stop_conditions.max_generations import MaxGenerations
9 import numpy as np
10
11 n_vars, pop_size = 20, 500
12
```

```

13 def mo_func(x):
14     f1 = np.sum(x)
15     f2 = np.sum(1 - x)
16     return np.array([f1, f2])
17
18 components = EDAComponents(
19     seeding      = RandomInit(),
20     selection    = CrowdingDistanceSelection(n_select=pop_size //
21     2),
22     learning     = LearnEBNA(),
23     sampling     = SampleBayesianNetwork(n_samples=pop_size),
24     replacement  = ElitistReplacement(n_elite=1),
25     stop_condition = MaxGenerations(50),
26 )
27 eda = EDA(pop_size, n_vars, mo_func,
28           np.full(n_vars, 2), components)
29 stats, cache = eda.run()

```

8.6 Extracting Pareto Fronts

After running the EDA with `cache_config` that stores populations and fitness values, the full Pareto set approximation can be extracted:

```

1 from pateda.selection.utils.pareto import find_pareto_set
2 import numpy as np
3
4 all_pop = np.vstack(cache.populations)
5 all_fit = np.vstack(cache.fitness_values)
6 pareto_idx = find_pareto_set(all_fit)    # indices of the non-
7     dominated set
8 pareto_pop = all_pop[pareto_idx]
9 pareto_fit = all_fit[pareto_idx]

```

9 Knowledge Extraction and Transfer Learning

One of the most distinctive features of pateda—inherited from MATEDA-2.0—is the ability to extract and visualize knowledge from the probabilistic models learned during the EDA search. The `knowledge_extraction` module provides tools organized into the following categories.

9.1 Dependency Analysis

`knowledge_extraction/dependency_analysis.py` provides:

`compute_correlation_matrix` Computes the Pearson correlation matrix between variables from a population.

`learn_bayesian_network` Learns a Bayesian network structure from a population using score-and-search.

`learn_gaussian_network` Learns a Gaussian graphical model from continuous data.

`analyze_variable_dependencies` Computes pairwise mutual information and tests for marginal independence.

`compute_mutual_information` Mutual information between two discrete random variables.

9.2 Fitness Measures

`knowledge_extraction/fitness_measures.py` provides tools to quantify the EDA’s behavior over generations:

Response to selection $R(t)$ $R(t) = \bar{f}(t+1) - \bar{f}(t)$: improvement in mean fitness from one generation to the next.

Amount of selection $S(t)$ $S(t) = \bar{f}_s(t) - \bar{f}(t)$: difference between the mean fitness of the selected subset and the whole population.

Realized heritability $b(t)$ $b(t) = R(t)/S(t)$: fraction of the selection differential that is realized in the next generation [53].

9.3 Network Measures and Visualizations

For models based on graphs (Bayesian networks, Markov networks, trees), pateda tracks which edges appear across generations and runs:

Edge frequency matrix [21, 72]. The (i, j) entry counts how often the edge (X_i, X_j) appeared in the structures learned across all runs and generations. Bright colors in the heat-map indicate frequent structural relationships, often corresponding to true problem-structure interactions.

Dendrogram of edges Hierarchical clustering of edges by co-occurrence frequency, displayed as a dendrogram [31].

Parallel coordinate view Each axis is a selected edge; each line connects edges that co-occur in the same structure at the same generation.

Glyph representation A grid of glyphs (one per run $\times$ generation) showing which edges are present in each learned structure.

Scalar structural descriptors of the learned graphs — degree distribution, clustering coefficient, betweenness, motif counts, and their evolution — are computed in `knowledge_extraction/network_measures.py` and plotted by the helpers in `network_visualizations.py` (`plot_edge_frequency_matrix`, `plot_measures_evolution`, `plot_network_snapshots`, `plot_degree_distribution`). Because `cache.models` stores the actual model learned at each generation (Section 3), all of these are produced from a single cached run.

Figures 4–6 illustrate the workflow on a concatenated order-3 deceptive function ($n = 15$, five blocks of three variables) optimized with EBNA. The single learned network (Figure 4) already recovers the true block structure: edges appear *within* the trap blocks and not across them. Accumulating the edges over all generations gives the edge-frequency matrix of Figure 5, whose block-diagonal pattern is a fingerprint of the problem’s true interactions. Finally, Figure 6 shows how the structure emerges along the run.

9.4 Continuous Variable Analysis

For continuous EDAs, `knowledge_extraction/continuous_visualizations.py` and `gaussian_networks.py` provide tools to visualize the models learned by Gaussian and Gaussian-network EDAs: `plot_gaussian_parameters` tracks the means, variances and correlations across generations; `plot_precision_heatmap` and `plot_partial_correlation_network` display the sparse conditional-independence structure recovered by a Gaussian-network (GMRf) EDA, where a zero in the precision matrix is a conditional independence; and `plot_network_comparison` contrasts two learned structures.

Bayesian network learned by EBNA (deceptive-3, blocks of 3 shaded)

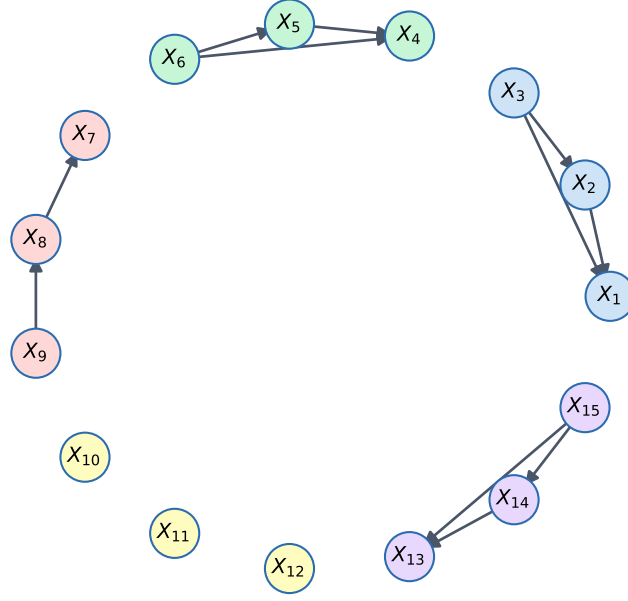


Figure 4: A Bayesian network learned by EBNA on the deceptive-3 problem. Nodes are shaded by their true order-3 block; the learned edges fall within blocks, recovering the problem structure.

For copula-based EDAs, the vine model is not a single covariance matrix but a *cascade of trees*, each level pairing (conditional) variables through a bivariate copula. The `vine_analysis.py` module together with `plot_vine_first_tree`, `plot_family_composition`, `plot_tau_by_tree` and `plot_vine_evolution` visualize this regular-vine decomposition: which pairs are linked at each tree level, which copula family (Gaussian, Clayton, Gumbel, ...) was selected for each edge, and the strength of dependence (Kendall's τ) per tree. Figure 7 shows the schematic structure of a regular vine over five variables.

9.5 Transfer Learning

The knowledge an EDA extracts about the structure of a problem need not be discarded when the run ends: it can be *transferred* to accelerate the solution of related problems. Transfer learning is attractive for EDAs precisely because they already build an explicit, inspectable model of the search distribution — the same edge-frequency and interaction information used for knowledge extraction above is exactly what a new run can be biased with. It is especially valuable when many similar instances must be solved (e.g. the same combinatorial problem over changing data) or when fitness evaluations are expensive and a warm start pays off [76, 33].

Following (author?) [76, 33], three kinds of transfer are relevant to EDAs:

Structural transfer Reuse of the learned *dependency structure*: the aggregated interaction/edge-frequency matrix of a source run becomes an a-priori **interaction_matrix** that restricts (or biases) structure learning in the target run — exactly the restriction mechanism used by TreeEDAR, MNFDAR and MNFDAGR (Section 10). This is the form validated on multi-marker tagging-SNP selection by (author?) [76].

Parametric transfer Reuse of *parameters/solutions*: seeding the target population from the source's best solutions or MPEs (**SeedThisPop**), i.e. a warm start.

Bias transfer A *soft* distance-based bias on the structure score that favours short-range dependencies learned to be useful on the source problem, as in the soft distance-based bias

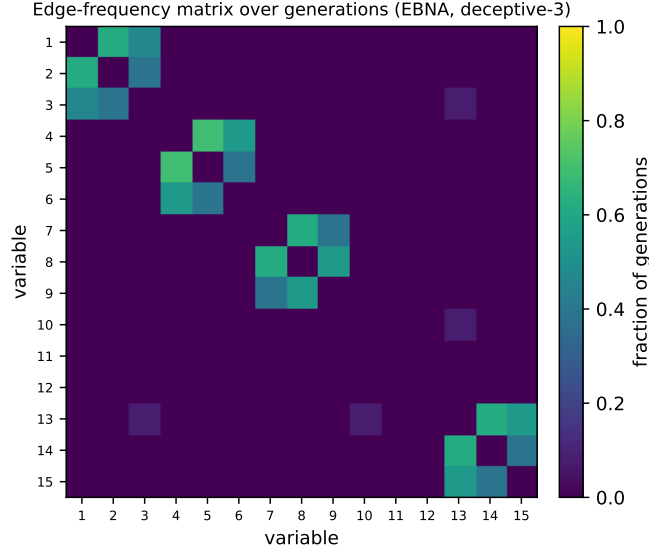


Figure 5: Edge-frequency matrix accumulated over generations for the same run. Each entry is the fraction of generations in which the corresponding (undirected) edge was present; the block-diagonal pattern matches the true interaction structure.

of (author?) [33].

These forms of transfer are supported by the `knowledge_extraction.transfer` module. `aggregate_edge_frequencies` averages the linkage graphs learned across a source run into an edge-frequency matrix; `structure_to_interaction_matrix` thresholds it into a binary `interaction_matrix` for hard structural transfer into TreeEDAR/MNFDAR/MNFDAGR. The `TransferredStructure` container packages this information, can save/load it to disk (so structure learned on one problem is reused on another), and exposes a `soft_bias()` edge prior for the soft-bias variant. Finally, `warm_start_population(...)` builds a warm-start population from a source run’s best solutions for parametric transfer via `SeedThisPop`.

```

1 from pateda.knowledge_extraction.transfer import TransferredStructure
2 ts = TransferredStructure.from_models(source_cache.models) #
   aggregate + persist
3 ts.save("problemA_structure.npz")
4 # ... later, on a related problem B:
5 M = TransferredStructure.load("problemA_structure.npz").
   interaction_matrix(0.5)
6 alg = TreeEDAR(n_vars=n, cardinality=2, fitness_func=fB,
   interaction_matrix=M)

```

10 Injecting A-Priori Knowledge

A key advantage of EDA-based approaches is that a-priori information about the problem structure can be directly incorporated into the model, potentially reducing the search space and accelerating convergence [74]. Because an EDA’s model *is* the search bias, partial knowledge of the problem — which variables interact, which values are plausible, which configurations are feasible — can be injected exactly where it matters instead of being encoded indirectly in operators. Restricting structure learning to a known interaction graph removes spurious edges that small selected sets would otherwise induce, and has been shown to improve both model quality and search on structured problems such as protein-structure prediction, side-chain placement and

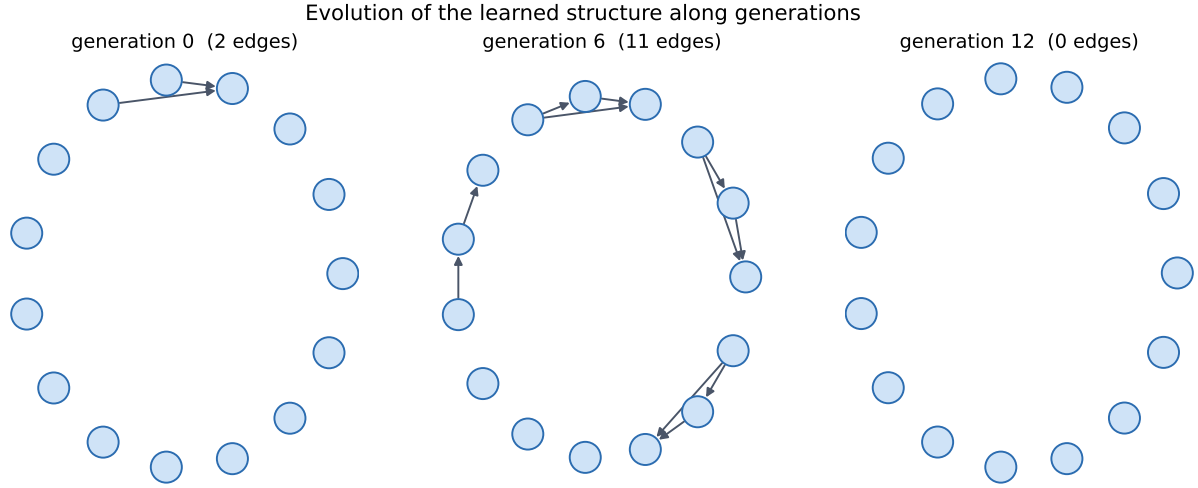


Figure 6: Evolution of the learned structure along the run (early, middle, and late generation). The model starts nearly empty and progressively concentrates edges inside the deceptive blocks.

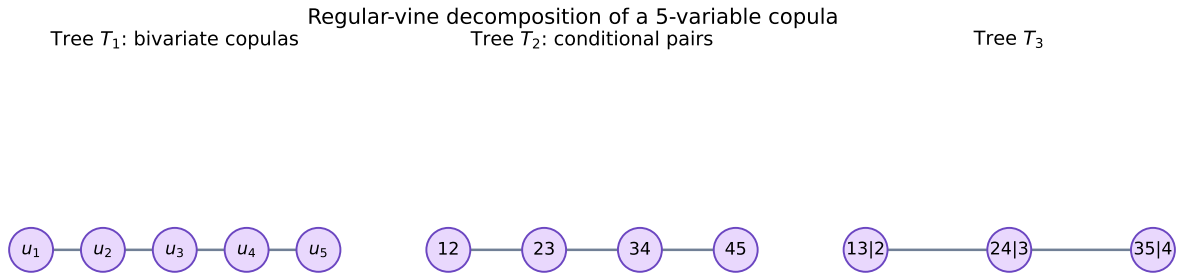


Figure 7: Regular-vine (R-vine) decomposition of a 5-variable copula. Tree T_1 pairs the variables through unconditional bivariate copulas; each subsequent tree T_k has the edges of T_{k-1} as its nodes and models conditional pairs (e.g. $13 | 2$), so the full joint density factorizes into a product of bivariate copulas.

tagging-SNP selection [72, 74, 76]. The mechanisms below — fixed factorizations, interaction matrices, biased initialization, and hard constraints — are the concrete entry points. Figure 8 illustrates the interaction-matrix mechanism.

10.1 Fixed Factorizations

When the structure of the optimal model is known a priori (e.g., from domain knowledge or problem decomposition), the user can supply a fixed clique structure to FDA:

```

1 import numpy as np
2 from pateda import FDA
3
4 # Fixed Markov-chain structure for a 10-variable problem
5 n = 10
6 cliques = np.array([
7     [0, 1, 0],          # factor 1: only  $X_1$ 

```

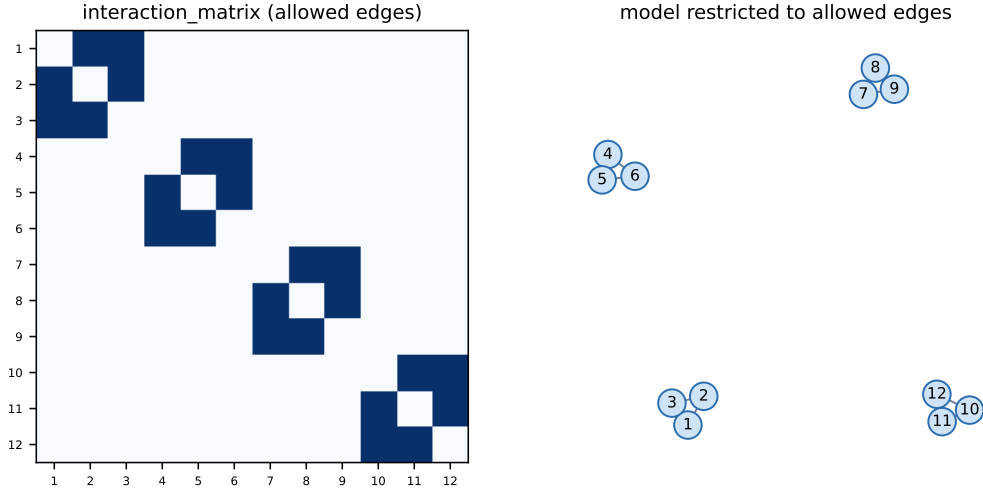


Figure 8: A-priori structural information as an interaction matrix. A binary matrix (left) lists the variable pairs allowed to interact; the restricted EDAs (**TreeEDAR**, **MNFDAR**, **MNFDAGR**) learn a model only over those edges (right), here a block-diagonal structure matching an additively decomposable problem.

```

8      [1, 1, 0, 1],      # factor 2:  $X_1$  /  $X_2$ 
9      [1, 1, 1, 2],      # factor 3:  $X_2$  /  $X_3$ 
10     # ... etc.
11 ] )
12
13 alg = FDA(n_vars=n, cardinality=2,
14           fitness_func=my_func,
15           cliques=cliques,
16           pop_size=300, n_gen=50)
17 stats, _ = alg.run()
```

The Markov-chain FDA with order 1 is the natural model for problems with sequential structure, such as the HP protein folding model [19, 69].

10.2 Interaction Matrices

TreeEDAR, **MNFDAR**, and **MNFDAGR** accept an `interaction_matrix` parameter: an $n \times n$ binary matrix where entry $(i, j) = 1$ means that variables X_i and X_j may be connected in the learned model, and $(i, j) = 0$ forbids the connection. This allows the user to encode known independence relationships or restrict the model to biologically or physically plausible interactions.

10.3 Biased Initialization

BiasInit seeds the initial population with variable-wise biases. For example, if domain knowledge indicates that variable X_5 is likely to take value 1 in good solutions, a higher prior probability can be assigned:

```

1 from pateda.seeding.bias_init import BiasInit
2
3 seeder = BiasInit(probs=np.array([0.5]*20 + [0.9]))
4 # Variable 21 is initialized with  $P(X=1) = 0.9$ 
```


10.4 Constrained Search

Most real optimization problems are *constrained*: only a subset of the representable strings are feasible. This is awkward for EDAs specifically because the model is learned from data and then sampled independently variable by variable: even if every selected (feasible) parent satisfies a global constraint, probabilistic logic sampling recombines their marginals and readily produces infeasible offspring. The three generic remedies — penalizing infeasibility in the fitness, repairing sampled solutions (Section on repairing methods), or building feasibility into the sampler — all apply in **pateda**; the third is the most reliable when it can be arranged.

A particularly clean case is the *unitation constraint*, which fixes the number of ones (the *unitation* $u(\mathbf{x}) = \sum_i x_i$) of every binary solution to a constant k . It appears whenever exactly k items must be chosen: feature selection with a fixed budget, fixed-size subset problems, balanced assignments. The feasible region is the set of length- n binary strings with exactly k ones, and independent univariate sampling almost never lands in it.

pateda provides two simple constraint-preserving EDAs for this case:

CUMDA [77]. Constrained UMDA. It learns univariate marginals as usual but *samples* with a scheme (stochastic universal sampling over the marginals) that emits exactly k ones per solution, so feasibility is maintained by construction; the seeding method likewise produces only feasible initial solutions.

CFDA [78]. Constrained FDA. Same fixed-unitation guarantee, but over a tree-structured factorized model rather than independent marginals, so it can additionally capture pairwise dependencies among the chosen positions.

Both are “simple” in the sense that they handle one specific, if common, family of constraints exactly; general constraints still require penalties or repair.

The weighting of selected solutions during model learning — another way of injecting fitness information into the model — is discussed with the learning machinery in Section 5.12.

11 Extensions to **pateda**

The component architecture of **pateda** was designed to be extended. Three specialized libraries take **pateda** as a dependency and reuse its executor, components, and statistics/knowledge machinery while replacing the representation or the model family. They are distributed separately so that their heavier dependencies (e.g. permutation-specific code, PyTorch) do not burden the core library.

11.1 Problems with permutation representation (**perm_pateda**)

perm_pateda⁶ targets optimization problems whose solutions are *permutations* (the travelling salesman problem, the linear ordering problem, flowshop scheduling, QAP). Permutations cannot be modelled variable-by-variable as integer vectors without producing infeasible offspring, so **perm_pateda** supplies permutation-specific learning and sampling components — distance-based probability models such as Mallows and generalized-Mallows models, and models based on the Plackett–Luce and factoradic representations — together with permutation seeding, repair, and benchmark problems. Everything else (the EDA loop, selection, replacement, stopping, statistics) is inherited from **pateda**.

⁶https://github.com/rsantana-isg/perm_pateda

11.2 Model learning with neural networks (nn_pateda)

`nn_pateda`⁷ replaces the probabilistic graphical model at the core of an EDA by a *neural* density/generative model. It provides learning/sampling components built on autoencoders and variational autoencoders, generative adversarial networks, and denoising/‘backdrive’ models that map fitness gradients back onto solutions [24, 52]. These plug into the same `learning/sampling` slots as the classical learners, so a neural EDA is assembled exactly like a UMDA or EBNA; PyTorch is used for the network components (with GPU support when available).

11.3 Searching in the neural latent representation space (latent_search)

`latent_search`⁸ implements search algorithms that first *learn a latent representation* of the optimization problem — typically with an autoencoder trained on good solutions — and then perform the search in that lower-dimensional, smoother latent space, decoding candidates back to the original space for evaluation. This “deep optimization” idea lets the algorithm discover and reuse building blocks captured by the representation, and connects EDAs to representation-learning approaches to black-box optimization.

12 EDA Pipelines: model operators and a generative grammar

An *EDA pipeline* is a consistent assembly of EDA components — seeding, selection, learning, sampling, replacement, optional local search and mutation, and a stopping condition — that together define a working EDA. The module `pateda.pipelines` adds two ingredients that make the space of pipelines both richer and searchable.

12.1 Model operators: MODMOD and MODCONV

Beyond the primary components of Section 6, two families of operators transform a learned probabilistic model (its graph *and* its tables):

MODMOD (model modifier) takes a model and returns a new model, usually simpler: `prune_factorized` reduces every clique of a factorized / junction model to at most K variables by marginalizing the dropped ones out of the factors; `tree_to_forest` cuts the weakest tree edges into a forest; `tree_to_malign` keeps only the *malign* edges (those whose interaction changes the conditional mode), a model-level Tree-EDA-M.

MODCONV (model conversor) changes the model *type* so a different sampler can consume it: `bn_to_factorized` turns a Bayesian network into an equivalent factorized model (one conditional clique per variable, in ancestral order), which the factorized samplers (`SampleFDA`, `SampleGibbs`) can sample.

The operator “LM MODCONV SM” of the design is realized by wrapping a learner in `ModifiedLearning(LM, [operators])`: the wrapper runs the base learner and applies the operator chain to its model, so that the resulting model NLM is samplable by SM. Every operator is robust (returns the model unchanged when inapplicable), so a modifier never breaks a pipeline. These operators substantially increase the number of feasible learning/sampling combinations: for example every Bayesian-network learner (EBNA, BOA, hBOA, LFDA, PADA) can now feed the factorized samplers.

⁷https://github.com/rsantana-isg/nn_pateda

⁸https://github.com/rsantana-isg/latent_search

12.2 A context-free grammar for feasible pipelines

Following the grammar-based AutoML literature [15, 51], `pateda.pipelines.grammar` defines a context-free grammar whose language is the set of valid discrete-EDA pipelines. The model part is organized into *typed blocks* (one per model type), so that a learner is always paired with a compatible sampler — directly, or through a MODCONV. A random derivation of the grammar is a flat list of terminals (each carrying a role and a factory) that `build_pipeline` reassembles into an EDA.

The top of the grammar is (“|” denotes choice, capitalized names are non-terminals):

```
1 Pipeline    -> Seeding Selection ModelBlock Replacement LocalOpt
      Mutation Stop
2 ModelBlock  -> FacBlock | BNBlock | MarkovNetBlock | IntFDABlock
3             | RegMarkovBlock | MarkovChainBlock
4 FacBlock    -> FacLearner FacModMod SampleFDA
5 BNBlock     -> BNLearner ( SampleBN | bn_to_factorized FacModMod
      SampleFDA )
6 FacModMod   -> empty | prune_factorized | tree_to_forest |
      tree_to_malign
```

The typed `ModelBlock` is the key to feasibility: each block pairs a family of learners with a sampler that can consume their model, and the optional model operators (`FacModMod`, and the `bn_to_factorized` MODCONV in `BNBlock`) are the bridges introduced above. Consequently *every* complete derivation is type-consistent by construction — a learner is never paired with an incompatible sampler. The residual fraction of random pipelines that still fail at run time comes only from data/representation edge cases (a binary-only mutation applied to a non-binary problem, a learner starved by a tiny population), which the accompanying demo measures explicitly rather than hiding.

The grammar’s generality is measured by the fraction of `pateda`’s discrete-EDA component implementations it can emit (per category: selection, learning, sampling, local optimization, mutation, replacement, ...), and its practicality by the fraction of randomly sampled pipelines that run without error; both are reported by the accompanying example.

12.3 A multi-objective meta-optimizer over pipelines

`PipelineMetaOptimizer` searches the space of pipelines defined by the grammar with a grammar-guided genetic algorithm. Each candidate is a whole pipeline, evaluated by building it and running it on a target problem at a *fixed inner budget* (population size and number of generations); this yields two conflicting objectives — the best objective value reached (to maximize) and the wall-clock time spent (to minimize). Because these objectives trade off, the search uses NSGA-II selection [18] and returns the *Pareto set* of pipelines, from the cheap-but-weaker to the strong-but-slower, rather than a single winner. The genetic operators respect the grammar: crossover swaps whole slots between parents — selection, replacement, local search, mutation, and the model block (learner + model operators + sampler) as a unit — and mutation re-derives one slot, so every offspring stays type-consistent. On a difficult, fully deceptive problem (a concatenated trap-5, where univariate models are fast but stall and linkage-learning models are slower but succeed) the meta-optimizer recovers the whole quality/time trade-off automatically, letting a practitioner pick the pipeline at the knee of the front for their time budget.

13 Conclusions

`pateda` is a comprehensive Python library for estimation of distribution algorithms over discrete and continuous, single- and multi-objective search spaces. It combines a set of plug-and-play algorithm classes with a fully modular component architecture — seeding, selection, learning,

sampling, replacement, mutation, crossover, local optimization, and repairing are independently swappable — and a knowledge-extraction module that turns the models learned during search into analyzable, visualizable descriptions of problem structure.

While retaining the modular philosophy of **MATEDA-2.0**, this version introduces a set of developments that go beyond it:

- support for discrete problems with heterogeneous (binary and non-binary) variable cardinalities;
- model learning that can explicitly use selection probabilities to weight solutions (Section 5.12), subsuming earlier fitness-weighted schemes;
- injection of a-priori problem-structure information, and its reuse across problems through transfer learning (Sections 10 and 9.5);
- an automatic EDA-pipeline search built on model operators (**MODMOD** / **MODCONV**) and a context-free grammar of feasible pipelines (Section 12);
- sampling based on the most probable configurations of the learned model (Section 5.7);
- a dedicated Bayesian-network learning library (**bayes_nets**) with a broad set of structure-learning algorithms, vine-copula models for continuous problems, and mixture models for discrete and continuous problems;
- three derived libraries — **perm_pateda**, **nn_pateda** and **latent_search** (Section 11) — that extend the framework to permutation problems, neural-network models, and search in learned latent spaces.

The developments collected in **ROADMAP_New_Functs.md** have been implemented in this version: diversity-preserving replacement schemes (crowding, restricted tournament, clustering), convergence- and stagnation-based stopping conditions, generalized probabilistic seeding, multi-objective metric tracking with a worked variant comparison, a continuous memetic example, and a first-class structural transfer-learning API. Further work will broaden the continuous-EDA family and add larger multi-objective and real-world benchmarks.

References

- [1] D. H. Ackley. An empirical study of bit vector function optimization. *Genetic Algorithms and Simulated Annealing*, pages 170–204, 1987.
- [2] J. E. Baker. Reducing bias and inefficiency in the selection algorithm. In *Proceedings of the Second International Conference on Genetic Algorithms*, pages 14–21. Lawrence Erlbaum Associates (Hillsdale), 1987.
- [3] S. Baluja. Population-based incremental learning: A method for integrating genetic search based function optimization and competitive learning. Technical Report CMU-CS-94-163, Carnegie Mellon University, Pittsburgh, PA, 1994.
- [4] S. Baluja. Incorporating a priori knowledge in probabilistic-model based optimization. In M. Pelikan, K. Sastry, and E. Cantú-Paz, editors, *Scalable Optimization via Probabilistic Modeling: From Algorithms to Applications*, Studies in Computational Intelligence, pages 205–222. Springer, 2006.

- [5] S. Baluja and S. Davies. Using optimal dependency-trees for combinatorial optimization: Learning the structure of the search space. In D. H. Fisher, editor, *Proceedings of the 14th International Conference on Machine Learning*, pages 30–38, San Francisco, CA., 1997. Morgan Kaufmann.
- [6] P. A. Bosman and J. Grahl. Matching inductive search bias and problem structure in continuous estimation of distribution algorithms. *European Journal of Operational Research*, 185:1246–1264, 2008.
- [7] P. A. Bosman and D. Thierens. Expanding from discrete to continuous estimation of distribution algorithms: The IDEA. In *Parallel Problem Solving from Nature - PPSN VI 6th International Conference*, Paris, France, September 16-20 2000. Springer. Lecture Notes in Computer Science 1917.
- [8] P. A. Bosman and D. Thierens. IDEAs based on the normal kernels probability density function. Technical Report UU-CS-2000-11, Utrecht University, 2000.
- [9] P. A. Bosman and D. Thierens. Advancing continuous IDEAs with mixture distributions and factorization selection metrics. In *Optimization by Building and Using Probabilistic Models (OBUPM) 2001*, pages 69–76, San Francisco, California, USA, 2001.
- [10] P. A. Bosman and D. Thierens. Multi-objective optimization with diversity preserving mixture-based iterated density estimation evolutionary algorithms. *International Journal of Approximate Reasoning*, 31(3):259–289, 2002.
- [11] D. Cho and B. Zhang. Evolutionary optimization by distribution estimation with mixtures of factor analyzer. In *Proceedings of the 2002 Congress on Evolutionary Computation CEC-2002*, volume 2, pages 1397–1401. IEEE press, 2002.
- [12] C. K. Chow and C. N. Liu. Approximating discrete probability distributions with dependence trees. *IEEE Transactions on Information Theory*, 14(3):462–467, 1968.
- [13] G. F. Cooper and E. Herskovits. A Bayesian method for the induction of probabilistic networks from data. *Machine Learning*, 9:309–347, 1992.
- [14] J. S. De Bonet, C. L. Isbell, and P. Viola. MIMIC: Finding optima by estimating probability densities. In M. C. Mozer, M. I. Jordan, and T. Petsche, editors, *Advances in Neural Information Processing Systems*, volume 9, pages 424–430. The MIT Press, Cambridge, 1997.
- [15] A. G. C. de Sá, W. J. G. S. Pinto, L. O. V. B. Oliveira, and G. L. Pappa. RECIPE: A grammar-based framework for automatically evolving classification pipelines. In *Genetic Programming (EuroGP 2017)*, volume 10196 of *Lecture Notes in Computer Science*, pages 246–261. Springer, 2017.
- [16] K. Deb. *Multi-objective Optimization Using Evolutionary Algorithms*. Wiley, Chichester, UK, 2001.
- [17] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.
- [18] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler. Scalable multi-objective optimization test problems. In *Proceedings of the 2002 Congress on Evolutionary Computation CEC-2002*, volume 2, pages 825–830. IEEE press, 2002.
- [19] K. A. Dill. Theory for the folding and stability of globular proteins. *Biochemistry*, 24(6):1501–1509, 1985.

- [20] C. Doerr, F. Ye, N. Horesh, H. Wang, O. M. Shir, and T. Baeck. Benchmarking discrete optimization heuristics with IOHprofiler. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 1798–1806, 2019.
- [21] C. Echegoyen, J. A. Lozano, R. Santana, and P. Larrañaga. Exact Bayesian network learning in estimation of distribution algorithms. In *Proceedings of the 2007 Congress on Evolutionary Computation CEC-2007*, pages 1051–1058. IEEE Press, 2007.
- [22] C. Echegoyen, R. Santana, J. A. Lozano, and P. Larrañaga. The impact of probabilistic learning algorithms in EDAs based on Bayesian networks. In *Linkage in Evolutionary Computation*, Studies in Computational Intelligence, pages 109–139. Springer, 2008.
- [23] R. Etxeberria and P. Larrañaga. Global optimization using Bayesian networks. In A. Ochoa, M. R. Soto, and R. Santana, editors, *Proceedings of the Second Symposium on Artificial Intelligence (CIMA-99)*, pages 332–339. Editorial Academia. Havana, Cuba, 1999.
- [24] U. Garciarena, R. Santana, and A. Mendiburu. Expanding variational autoencoders for learning and exploiting latent representations in search distributions. In *Proceedings of the 2018 on Genetic and Evolutionary Computation Conference*, pages 849–856. ACM, 2018.
- [25] D. E. Goldberg. Simple genetic algorithms and the minimal, deceptive problem. In L. Davis, editor, *Genetic Algorithms and Simulated Annealing*, pages 74–88. Pitman Publishing, London, UK, 1987.
- [26] D. E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA, 1989.
- [27] J. M. Hammersley and P. Clifford. Markov fields of finite graphs and lattice. Technical report, University of California-Berkeley, 1968.
- [28] P. Hansen and N. Mladenović. Variable neighborhood search. In F. Glover and G. A. Kochenberger, editors, *Handbook of Metaheuristics*, pages 145–184. Springer, 2003.
- [29] G. Harik. Linkage learning via probabilistic modeling in the ECGA. IlliGAL Report 99010, University of Illinois at Urbana-Champaign, Illinois Genetic Algorithms Laboratory, Urbana, IL, 1999.
- [30] G. R. Harik. Finding multimodal solutions using restricted tournament selection. In *Proceedings of the 6th International Conference on Genetic Algorithms (ICGA)*, pages 24–31, 1995.
- [31] M. Hauschild and M. Pelikan. Enhancing efficiency of hierarchical BOA via distance-based model restrictions. MEDAL Report No. 2008007, Missouri Estimation of Distribution Algorithms Laboratory (MEDAL), April 2008.
- [32] M. Hauschild and M. Pelikan. Network crossover performance on NK landscapes and deceptive problems. Technical Report 2010003, Missouri Estimation of Distribution Algorithms Laboratory (MEDAL), 2010. Also in Proceedings of GECCO-2010.
- [33] M. W. Hauschild and M. Pelikan. Transfer learning, soft distance-based bias, and the hierarchical BOA. In *Parallel Problem Solving from Nature – PPSN XII*, volume 7491 of *Lecture Notes in Computer Science*, pages 173–183. Springer, 2012.
- [34] J. H. Holland. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. University of Michigan Press, Ann Arbor, MI, 1975.

- [35] I. Inza, P. Larrañaga, R. Etxeberria, and B. Sierra. Feature subset selection by Bayesian network-based optimization. *Artificial Intelligence*, 123(1-2):157–184, 2000.
- [36] A. Johnson and J. L. Shapiro. The importance of selection mechanisms in distribution estimation algorithms. In P. Collet, editor, *Proceedings of EA 2001*, volume 2310 of *Lecture Notes in Computer Science*, pages 91–103. Springer, 2002.
- [37] S. Kauffman. *Origins of Order*. Oxford University Press, 1993.
- [38] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [39] D. Koller and N. Friedman. *Probabilistic Graphical Models: Principles and Techniques*. The MIT Press, 2009.
- [40] P. Larrañaga. *Estimation of Distribution Algorithms. A New Tool for Evolutionary Computation*, chapter An introduction to probabilistic graphical models, pages 25–54. Kluwer Academic Publishers, Boston/Dordrecht/London, 2002.
- [41] P. Larrañaga, R. Etxeberria, J. A. Lozano, and J. Peña. Combinatorial optimization by learning and simulation of Bayesian networks. In *Proceedings of the Sixteenth Annual Conference on Uncertainty in Artificial Intelligence (UAI-2000)*, pages 343–352, San Francisco, CA, 2000. Morgan Kaufmann Publishers.
- [42] P. Larrañaga, R. Etxeberria, J. A. Lozano, and J. M. Peña. Optimization by learning and simulation of Bayesian and Gaussian networks. Technical Report EHU-KZAA-IK-4/99, Department of Computer Science and Artificial Intelligence, University of the Basque Country, 1999.
- [43] P. Larrañaga and J. A. Lozano, editors. *Estimation of Distribution Algorithms. A New Tool for Evolutionary Computation*. Kluwer Academic Publishers, Boston/Dordrecht/London, 2002.
- [44] S. Lauritzen and D. J. Spiegelhalter. Local computations with probabilities on graphical structures and their application to expert systems (with discussion). *Journal of the Royal Statistical Society, Series B*, 50:157–224, 1988.
- [45] S. L. Lauritzen. *Graphical Models*. Oxford Clarendon Press, 1996.
- [46] C. F. Lima, M. Pelikan, D. E. Goldberg, F. G. Lobo, K. Sastry, and M. Hauschild. Influence of selection and replacement strategies on linkage learning in BOA. In *Proceedings of the 2007 Congress on Evolutionary Computation CEC-2007*, pages 1083–1090. IEEE Press, 2007.
- [47] C. F. Lima, M. Pelikan, K. Sastry, M. Butz, D. E. Goldberg, and F. G. Lobo. Substructural neighborhoods for local search in the Bayesian optimization algorithm. In *Parallel Problem Solving from Nature - PPSN IX*, volume 4193 of *Lecture Notes in Computer Science*, pages 232–241. Springer, 2006.
- [48] J. A. Lozano, P. Larrañaga, I. Inza, and E. Bengoetxea, editors. *Towards a New Evolutionary Computation: Advances on Estimation of Distribution Algorithms*. Springer, 2006.
- [49] S. W. Mahfoud. *Niching methods for genetic algorithms*. PhD thesis, University of Illinois at Urbana-Champaign, Urbana, IL, May 1995. Also IlliGAL Report No. 95001.
- [50] T. Mahnig and H. Mühlenbein. Comparing the adaptive Boltzmann selection schedule SDS to truncation selection. In *Evolutionary Computation and Probabilistic Graphical Models. Proceedings of the Third Symposium on Adaptive Systems (ISAS-2001)*, pages 121–128, Havana, Cuba, March 2001.

- [51] R. Marinescu, A. Kishimoto, P. Ram, A. Rawat, M. Wistuba, P. P. Palmes, and A. Botea. Searching for machine learning pipelines using a context-free grammar. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 8902–8911, 2021.
- [52] L. Marti, J. García, A. Berlanga, and J. M. Molina. Introducing MONEDA: scalable multiobjective optimization with a neural estimation of distribution algorithm. In *Proceedings of the 10th Annual Conference on Genetic and Evolutionary Computation GECCO-2008*, pages 689–696, New York, NY, USA, 2008. ACM.
- [53] H. Mühlenbein. The equation for response to selection and its use for prediction. *Evolutionary Computation*, 5(3):303–346, 1997.
- [54] H. Mühlenbein and T. Mahnig. FDA – a scalable evolutionary algorithm for the optimization of additively decomposed functions. *Evolutionary Computation*, 7(4):353–376, 1999.
- [55] H. Mühlenbein, T. Mahnig, and A. Ochoa. Schemata, distributions and graphical models in evolutionary optimization. *Journal of Heuristics*, 5(2):213–247, 1999.
- [56] H. Mühlenbein and G. Paaß. From recombination of genes to the estimation of distributions I. Binary parameters. In H.-M. Voigt, W. Ebeling, I. Rechenberg, and H.-P. Schwefel, editors, *Parallel Problem Solving from Nature - PPSN IV*, volume 1141 of *Lectures Notes in Computer Science*, pages 178–187, Berlin, 1996. Springer.
- [57] M. Nomura and M. Shibata. cmaes: A simple yet practical python library for cma-es. *CoRR*, abs/2402.01373, 2024.
- [58] A. Ochoa, H. Mühlenbein, and M. Soto. Factorized distribution algorithms using Bayesian networks bounded complexity. In *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-2000*, pages 212–215, 2000.
- [59] A. Ochoa, M. R. Soto, R. Santana, J. Madera, and N. Jorge. The factorized distribution algorithm and the junction tree: A learning perspective. In A. Ochoa, M. R. Soto, and R. Santana, editors, *Proceedings of the Second Symposium on Artificial Intelligence (CIMA-99)*, pages 368–377, Havana, Cuba, March 1999. Editorial Academia. Havana, Cuba.
- [60] M. Pelikan. *Hierarchical Bayesian Optimization Algorithm. Toward a New Generation of Evolutionary Algorithms*, volume 170 of *Studies in Fuzziness and Soft Computing*. Springer, 2005.
- [61] M. Pelikan and D. E. Goldberg. Escaping hierarchical traps with competent genetic algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-2001*, pages 511–518. Morgan Kaufmann, 2001.
- [62] M. Pelikan, D. E. Goldberg, and E. Cantú-Paz. BOA: The Bayesian optimization algorithm. In W. Banzhaf, J. Daida, A. E. Eiben, M. H. Garzon, V. Honavar, M. Jakiela, and R. E. Smith, editors, *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-1999*, volume I, pages 525–532, Orlando, FL, 1999. Morgan Kaufmann Publishers, San Francisco, CA.
- [63] M. Pelikan and H. Mühlenbein. The bivariate marginal distribution algorithm. In R. Roy, T. Furuhashi, and P. Chawdhry, editors, *Advances in Soft Computing - Engineering Design and Manufacturing*, pages 521–535, London, 1999. Springer.
- [64] M. Pelikan, K. Sastry, and E. Cantú-Paz, editors. *Scalable Optimization via Probabilistic Modeling: From Algorithms to Applications*. Studies in Computational Intelligence. Springer, 2006.

- [65] E. Radetic, M. Pelikan, and D. E. Goldberg. Effects of a deterministic hill climber on hBOA. In *Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation (GECCO-2009)*, pages 437–444, Montréal, Québec, Canada, 2009. ACM.
- [66] R. Santana. A Markov network based factorized distribution algorithm for optimization. In *Proceedings of the 14th European Conference on Machine Learning (ECML-PKDD 2003)*, volume 2837 of *Lecture Notes in Artificial Intelligence*, pages 337–348, Dubrovnik, Croatia, 2003. Springer.
- [67] R. Santana. Estimation of distribution algorithms with Kikuchi approximations. *Evolutionary Computation*, 13(1):67–97, 2005.
- [68] R. Santana, H. Karshenas, C. Bielza, and P. Larrañaga. Regularized k -order Markov models in EDAs. In *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation (GECCO-2011)*, pages 593–600, Dublin, Ireland, 2011. ACM.
- [69] R. Santana, P. Larrañaga, and J. A. Lozano. Protein folding in 2-dimensional lattices with estimation of distribution algorithms. In *Proceedings of the First International Symposium on Biological and Medical Data Analysis*, volume 3337 of *Lecture Notes in Computer Science*, pages 388–398, Barcelona, 2004. Springer.
- [70] R. Santana, P. Larrañaga, and J. A. Lozano. Interactions and dependencies in estimation of distribution algorithms. In *Proceedings of the 2005 Congress on Evolutionary Computation (CEC-2005)*, pages 1418–1425, Edinburgh, U.K., 2005. IEEE Press.
- [71] R. Santana, P. Larrañaga, and J. A. Lozano. Interactions and dependencies in estimation of distribution algorithms. In *Proceedings of the 2005 Congress on Evolutionary Computation CEC-2005*, pages 1418–1425, Edinburgh, U.K., 2005. IEEE Press.
- [72] R. Santana, P. Larrañaga, and J. A. Lozano. The role of a priori information in the minimization of contact potentials by means of estimation of distribution algorithms. In E. Marchiori, J. H. Moore, and J. C. Rajapakse, editors, *Proceedings of the Fifth European Conference on Evolutionary Computation, Machine Learning and Data Mining in Bioinformatics*, volume 4447 of *Lecture Notes in Computer Science*, pages 247–257, Valencia, Spain, 2007. Springer.
- [73] R. Santana, P. Larrañaga, and J. A. Lozano. Protein folding in simplified models with estimation of distribution algorithms. *IEEE Transactions on Evolutionary Computation*, 12(4):418–438, 2008.
- [74] R. Santana, P. Larrañaga, and J. A. Lozano. Research topics on discrete estimation of distribution algorithms. *Memetic Computing*, 1(1):35–54, 2009.
- [75] R. Santana, P. Larrañaga, and J. A. Lozano. Learning factorizations in estimation of distribution algorithms using affinity propagation. *Evolutionary Computation*, 18(4):515–546, 2010.
- [76] R. Santana, A. Mendiburu, and J. A. Lozano. Structural transfer using EDAs: An application to multi-marker tagging SNP selection. In *2012 IEEE Congress on Evolutionary Computation (CEC)*, pages 1–8. IEEE, 2012.
- [77] R. Santana and A. Ochoa. Dealing with constraints with estimation of distribution algorithms: The univariate case. In A. Ochoa, M. R. Soto, and R. Santana, editors, *Proceedings of the Second Symposium on Artificial Intelligence (CIMA-99)*, pages 378–384, Havana, Cuba, March 1999.

- [78] R. Santana, A. Ochoa, and M. R. Soto. Factorized Distribution Algorithms for functions with unication constraints. In *Evolutionary Computation and Probabilistic Graphical Models. Proceedings of the Third Symposium on Adaptive Systems (ISAS-2001)*, pages 158–165, Havana, Cuba, March 2001.
- [79] R. Santana, A. Ochoa, and M. R. Soto. The mixture of trees factorized distribution algorithm. In L. Spector, E. Goodman, A. Wu, W. Langdon, H. Voigt, M. Gen, S. Sen, M. Dorigo, S. Pezeshk, M. Garzon, and E. Burke, editors, *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-2001*, pages 543–550, San Francisco, CA, 2001. Morgan Kaufmann Publishers.
- [80] R. Santana, A. Ochoa, and M. R. Soto. The mixture of trees factorized distribution algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-2001*, pages 543–550, San Francisco, CA, 2001. Morgan Kaufmann Publishers.
- [81] R. Santana, A. Ochoa, and M. R. Soto. Solving problems with integer representation using a tree based Factorized Distribution Algorithm. In *Proceedings of the First International NAISO Congress on Neuro Fuzzy Technologies*, Havana, Cuba, 2002.
- [82] G. Schwarz. Estimating the dimension of a model. *Annals of Statistics*, 7(2):461–464, 1978.
- [83] B. Selman, H. Levesque, and D. Mitchell. A new method for solving hard satisfiability problems. In C. American Association for Artificial Intelligence, editor, *Proceedings of the 10th National Conference on Artificial Intelligence (AAAI-92)*, pages 440–446, San Jose, CA, USA, 1992. American Association for Artificial Intelligence. Menlo Park, CA, USA.
- [84] S. Shakya and J. McCall. Optimization by estimation of distribution with DEUM framework based on Markov random fields. *International Journal of Automation and Computing*, 4(3):262–272, 2007.
- [85] S. Shakya and R. Santana. An EDA based on local Markov property and Gibbs sampling. In M. Keijzer, editor, *Proceedings of the 2008 Genetic and Evolutionary Computation Conference (GECCO)*, pages 475–476, New York, NY, USA, 2008. ACM.
- [86] S. Shakya, R. Santana, and J. A. Lozano. A Markovianity based optimisation algorithm. *Genetic Programming and Evolvable Machines*, 13(2):159–195, 2012.
- [87] V. P. Soloviev, P. Larrañaga, and C. Bielza. EDAspy: An extensible python package for estimation of distribution algorithms. *Neurocomputing*, 598:128043, 2024.
- [88] M. R. Soto, A. Ochoa, S. Acid, and L. M. Campos. Bayesian evolutionary algorithms based on simplified models. In A. Ochoa, M. R. Soto, and R. Santana, editors, *Proceedings of the Second Symposium on Artificial Intelligence (CIMAF-99)*, pages 360–367, Havana, Cuba, March 1999.
- [89] D. Vermetten, J. Rook, O. L. Preuß, J. de Nobel, C. Doerr, M. López-Ibañez, H. Trautmann, and T. Bäck. MO-IOHinspector: Anytime benchmarking of multi-objective algorithms using IOHprofiler. *CoRR*, abs/2412.07444, 2024.
- [90] D. Yazdani, M. N. Omidvar, D. Yazdani, K. Deb, and A. H. Gandomi. GNBG: A generalized and configurable benchmark generator for continuous numerical optimization. *CoRR*, abs/2312.07083, 2023.
- [91] M. Zangari, A. Mendiburu, R. Santana, and A. Pozo. Multiobjective decomposition-based Mallows models estimation of distribution algorithm. A case of study for permutation flow-shop scheduling problem. *Information Sciences*, 397–398:137–154, 2017.

- [92] M. Zangari, A. Pozo, R. Santana, and A. Mendiburu. A decomposition-based binary ACO algorithm for the multiobjective UBQP. *Neurocomputing*, 246:58–68, 2017.
- [93] Q. Zhang and H. Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007.
- [94] H. Zou and T. Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2):301–320, 2005.

Class	Identifier	Description
UMDA	UMDA	Univariate marginal distribution algorithm [56]
PBIL	PBIL	Population-based incremental learning [3]
MIMIC	MIMIC	Mutual-information maximization for input clustering [14]
BMDA	BMDA	Bivariate marginal distribution algorithm [63]
TreeEDA	Tree-EDA	Maximum-weight spanning tree of pairwise MI [5, 63]
TreeEDAR	Tree-EDA_r	Tree-EDA with restricted interaction matrix [4, 72]
TreeEDAM	Tree-EDA-M	Tree-EDA keeping only malign interactions [70]
IntFDA	Int_FDA	Tree-based FDA for high-cardinality integers [81]
AffEDA	AffEDA	Affinity-propagation cliques [75]
MKEDA	MK-EDA k	k -order Markov chain EDA [69]
MkRgEDA	MkRg-EDA	Regularized k -order Markov EDA [68]
MTED	MT-EDA	Mixture of trees EDA [80]
MNFDA	MN-FDA	Markov network factorization from χ^2 dependency tests [66]
MNF DAR	MN-FDA_r	MN-FDA restricted to an a-priori interaction matrix [67]
MNF DAG	MN-FDA g	Markov network factorization from G -test dependency tests [67]
MNF DAGR	MN-FDA g _r	MN-FDA g restricted to an a-priori interaction matrix [67]
MOA	MOA	Markovianity-based optimization algorithm [85, 86]
EBNA	EBNA	Estimation of Bayesian network algorithm [23]
BOA	BOA	Bayesian optimization algorithm [62]
HBOA	HBOA	Hierarchical Bayesian optimization algorithm [61, 60]
PADA	PADA	Polytree approximation of distribution algorithm [88, 58]
FDA	FDA	Factorized distribution algorithm [55, 59]
LFDA	LFDA	FDA with a learned Bayesian network factorization [54]
SARTRE_EDA	SARTRE-EDA	BN-EDA with sparse-additive (group-lasso) structure learning [?]
BINOTEARS_EDA	BINOTEARS-EDA	BN-EDA with differentiable structure learning; binary only [?]
PCBN_EDA	PCBN-EDA	BN-EDA with bounded-order constraint-based (PC-Stable) learning [?]
HSARTRE_EDA	HSARTRE-EDA	SARTRE-EDA with decision-graph local structure and niching [60]
HBINOTEARS_EDA	HBINOTEARS-EDA	BINOTEARS-EDA with decision-graph local structure and niching; binary only [60]
CUMDA	CUMDA	Unitation-constrained UMDA [77]
CFDA	CFDA	Unitation-constrained FDA [78]

Table 1: Discrete plug-and-play algorithm classes in pateda.

Class	Identifier	Description
<code>GaussianUMDA</code>	G-UMDA	Univariate Gaussian marginals [42, 41]
<code>GaussianFull</code>	G-Full	Full multivariate Gaussian [41, 8]
<code>GaussianNetworkEDA</code>	G-Net	Gaussian network (GMRF) EDA
<code>MixtureGaussianEDA</code>	MGaussian	Mixture of Gaussians [10]

Table 2: Continuous plug-and-play algorithm classes in pateda.